

Computergrafik

PD Prof. Dr. rer nat.
Marco Block-Berlitz

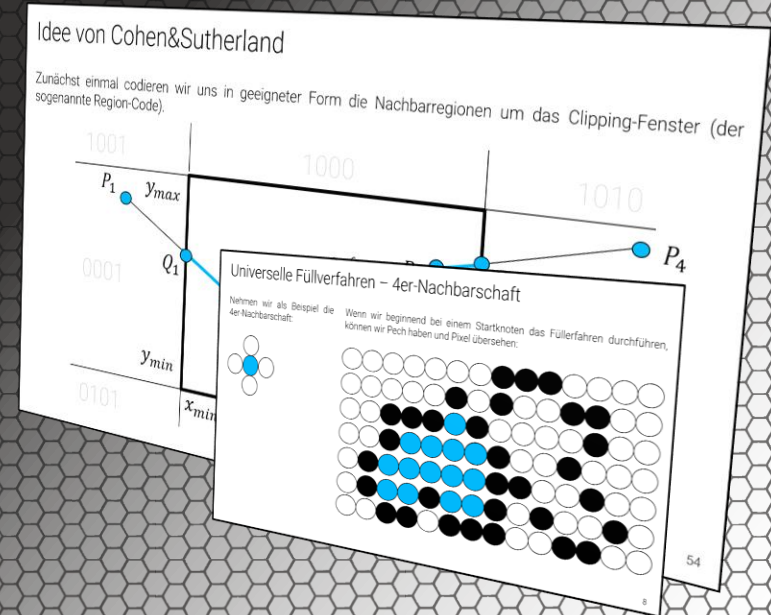
Sommersemester 2026



Rasterisierung von Flächen und Clipping von Linien und Polygonen

„Nichts ist so beständig wie der Wandel.“ (Heraklit von Ephesus)

PD Prof. Dr. Marco Block-Berlitz



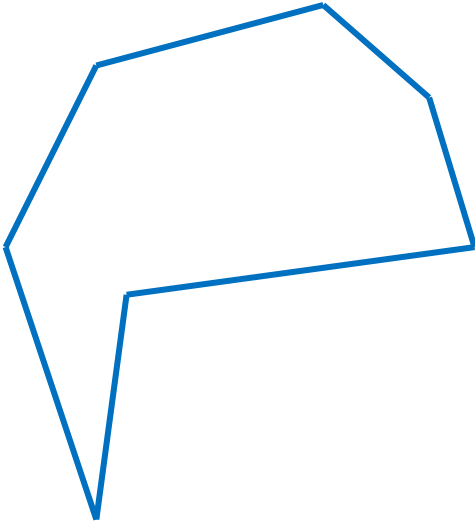
Ziele und Inhalt

- Universelle Füllverfahren mit 4er-Nachbarschaft
 - Universelle Füllverfahren mit 8er-Nachbarschaft
 - Scan-Line-Verfahren
 - Betrachtung der Spezialfälle
 - Schnittpunkte vorhersagen
 - Kreisfüllen mit Scan-Line-Verfahren
 - Motivation zu Clipping
 - Clipping von Linien
 - Region-Code von Cohen&Sutherland
 - Schnittpunkte direkt berechnen
 - Clipping von Polygonen
 - Lokale Betrachtung
 - Idee von Sutherland&Hodgman
- Fragestellungen zum Vorlesungsteil
 - Praktische Aufgaben



2D-Flächen füllen

Wir können einfache und komplexe 2D-Formen konstruieren. Wir betrachten nun [rasterbasierte Verfahren](#) zur Füllung von Polygonflächen.



Polygon



Polygon gefüllt

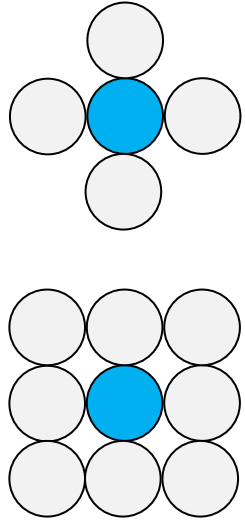




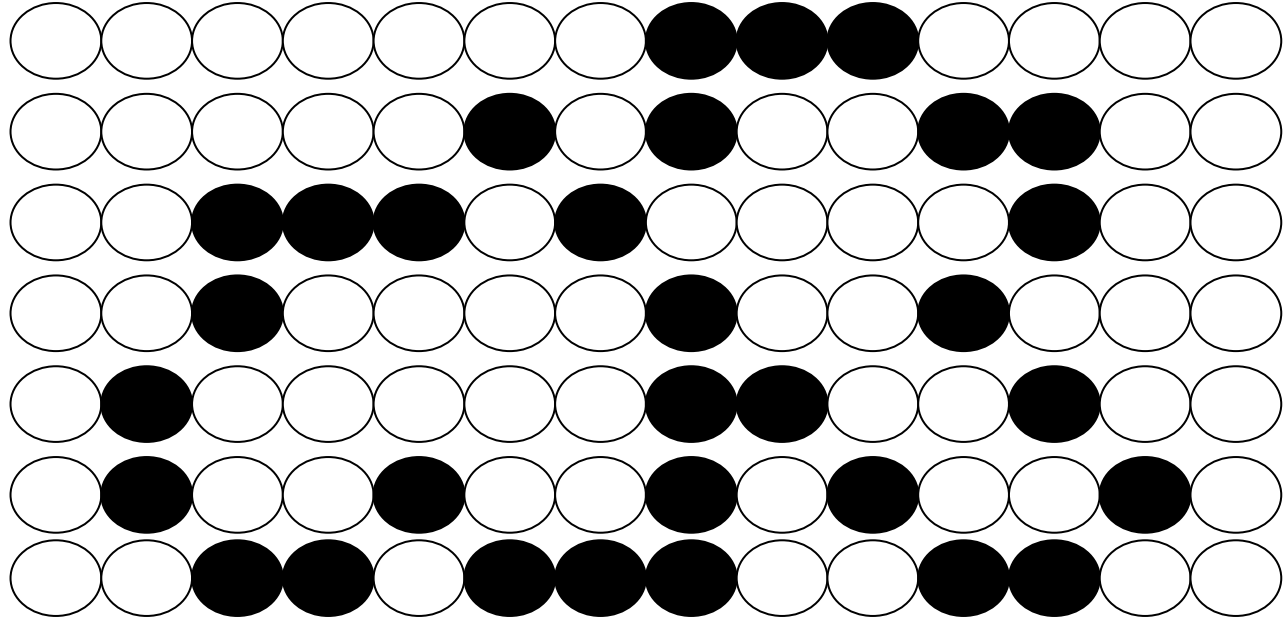
Flood-Fill-Algorithmen: Schauen wir uns eine einfache Strategie zum Polygonfüllen an

Universelle Füllverfahren – Flood-Fill-Algorithmen

Wir definieren Nachbarschaften, wie z. B. die **4er- oder 8er-Nachbarschaft**:

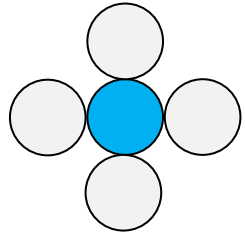


Wenn wir uns in einer endlichen Fläche befinden können wir die Zusammenhangeigenschaften ausnutzen:

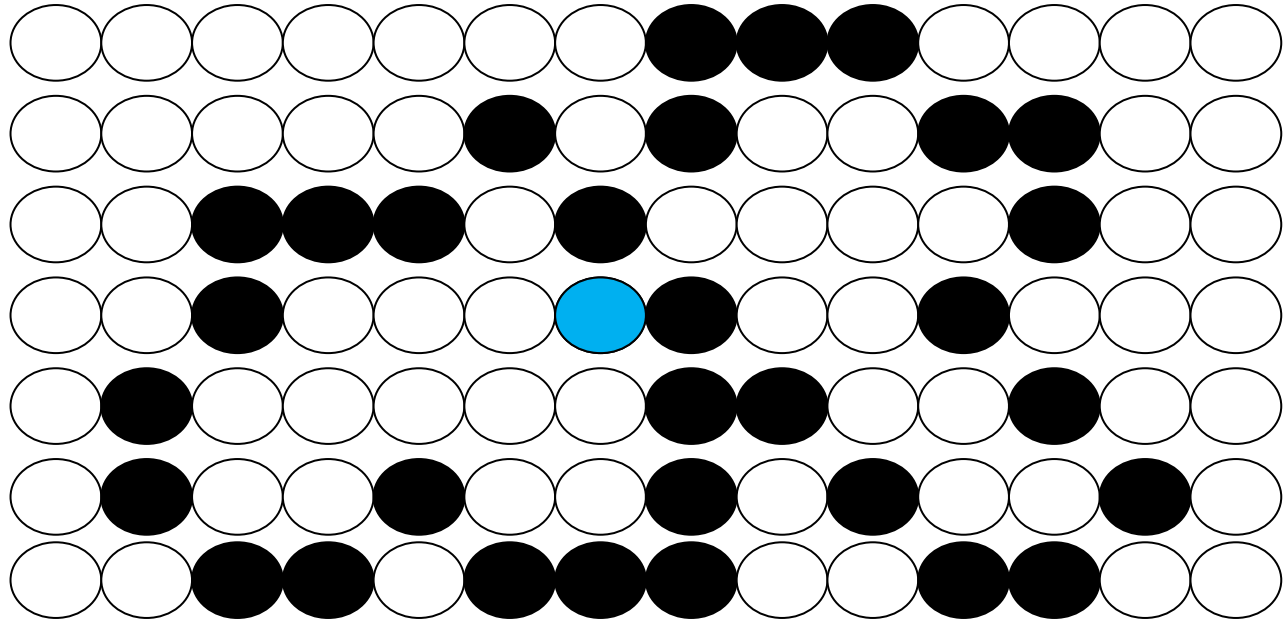


Universelle Füllverfahren – 4er-Nachbarschaft

Nehmen wir als Beispiel die 4er-Nachbarschaft:

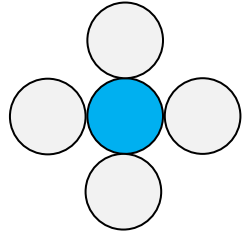


Wenn wir beginnend bei einem Startknoten das Füllerfahren durchführen, können wir Pech haben und Pixel übersehen:

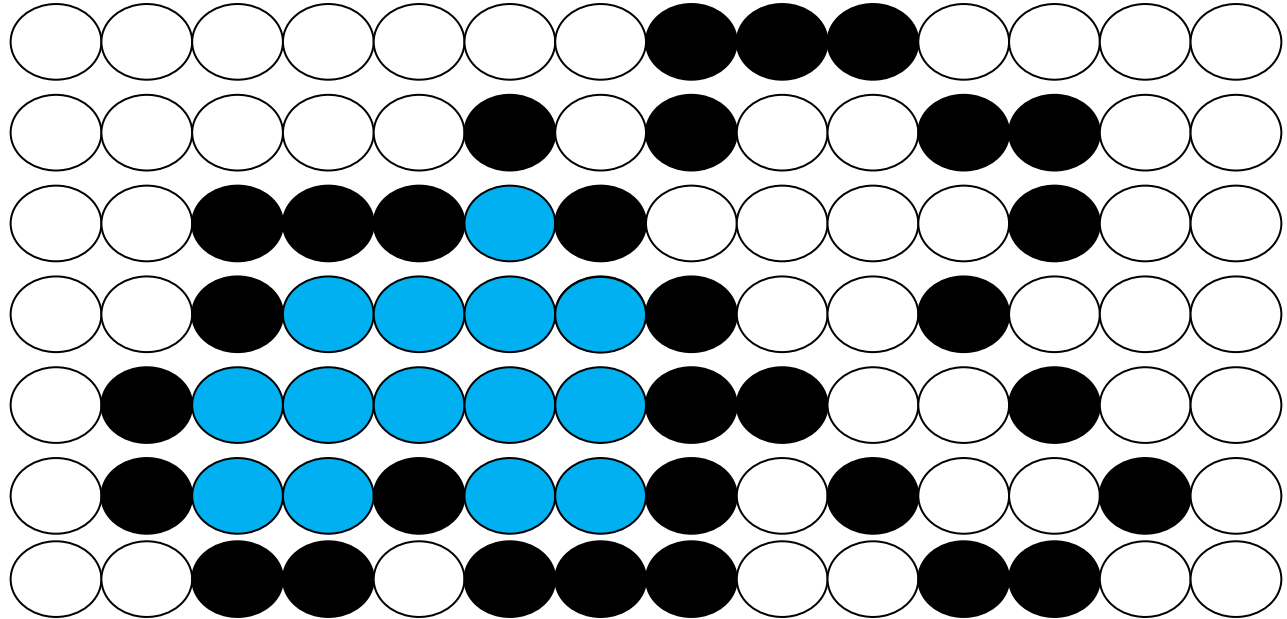


Universelle Füllverfahren – 4er-Nachbarschaft

Nehmen wir als Beispiel die 4er-Nachbarschaft:

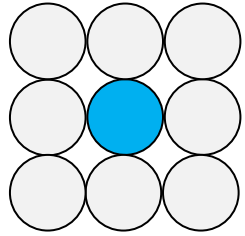


Wenn wir beginnend bei einem Startknoten das Füllerfahren durchführen, können wir Pech haben und Pixel übersehen:

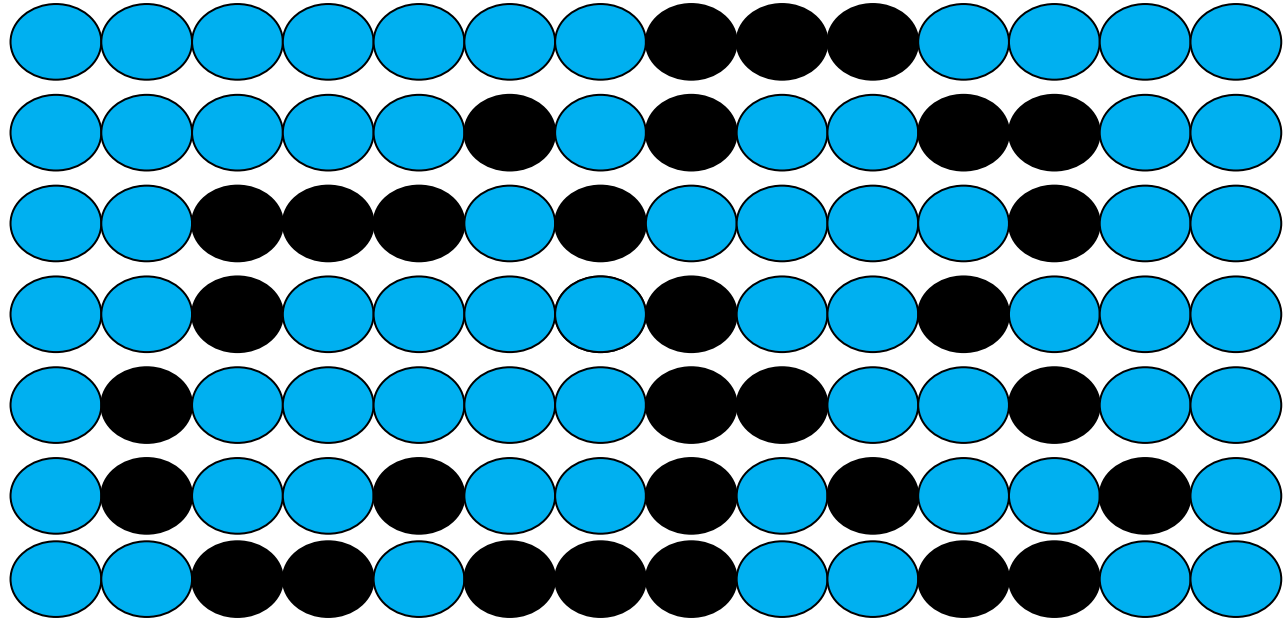


Universelle Füllverfahren – 8er-Nachbarschaft

Nehmen wir als Beispiel die 8er-Nachbarschaft:



Wenn wir beginnend bei einem Startknoten das Füllerfahren durchführen, können wir Pech haben und aus der Fläche springen:



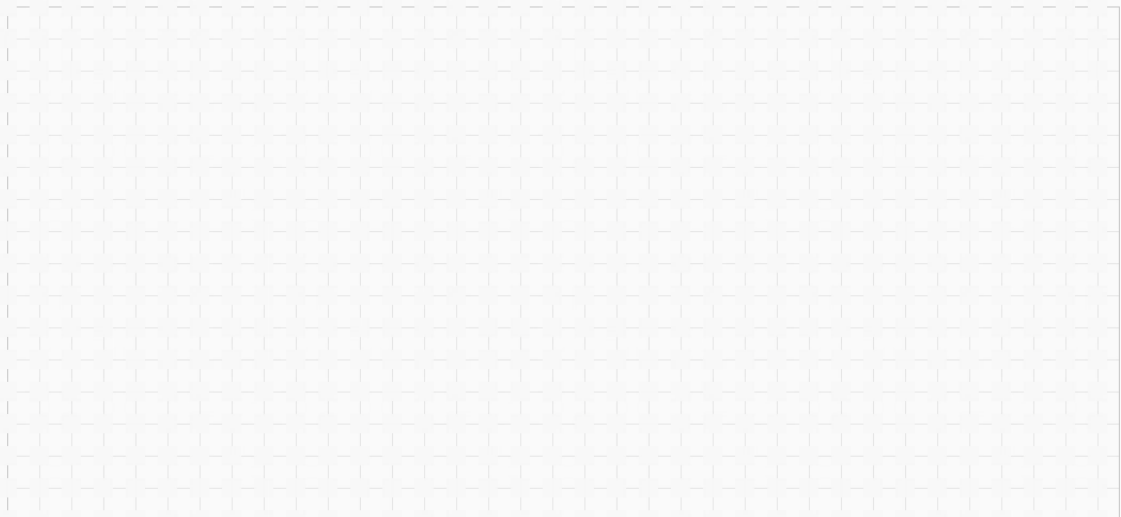
Implementierungsbeispiel – Naiver Ansatz

Schauen wir uns hier das Beispiel in einer interaktiven Umgebung an. Es kann ein Polygon definiert und die Nachbarschaft ausgewählt werden.

Danach können wir den Flood-Fill-Ansatz schrittweise durchführen.

Polygon: Klicke Gitterpunkte. Schließen durch Klick auf P0 (mind. 3 Punkte).
Danach: 4 = 4er-Kernel, 8 = 8er-Kernel, Klick auf freie Zelle = Seed setzen.
SPACE = nächster Schritt, B = zurück, A = Autoplay, R = kompletter Reset
Kernelwechsel leert nur die Füllung, das Polygon bleibt bestehen.

Modus: Polygon zeichnen
Kernel: 4er
Seed: -
Schritte: 0 | Queue: 0



Legende:

■ Polygonrand ■ Queue / Frontier ■ gefüllt

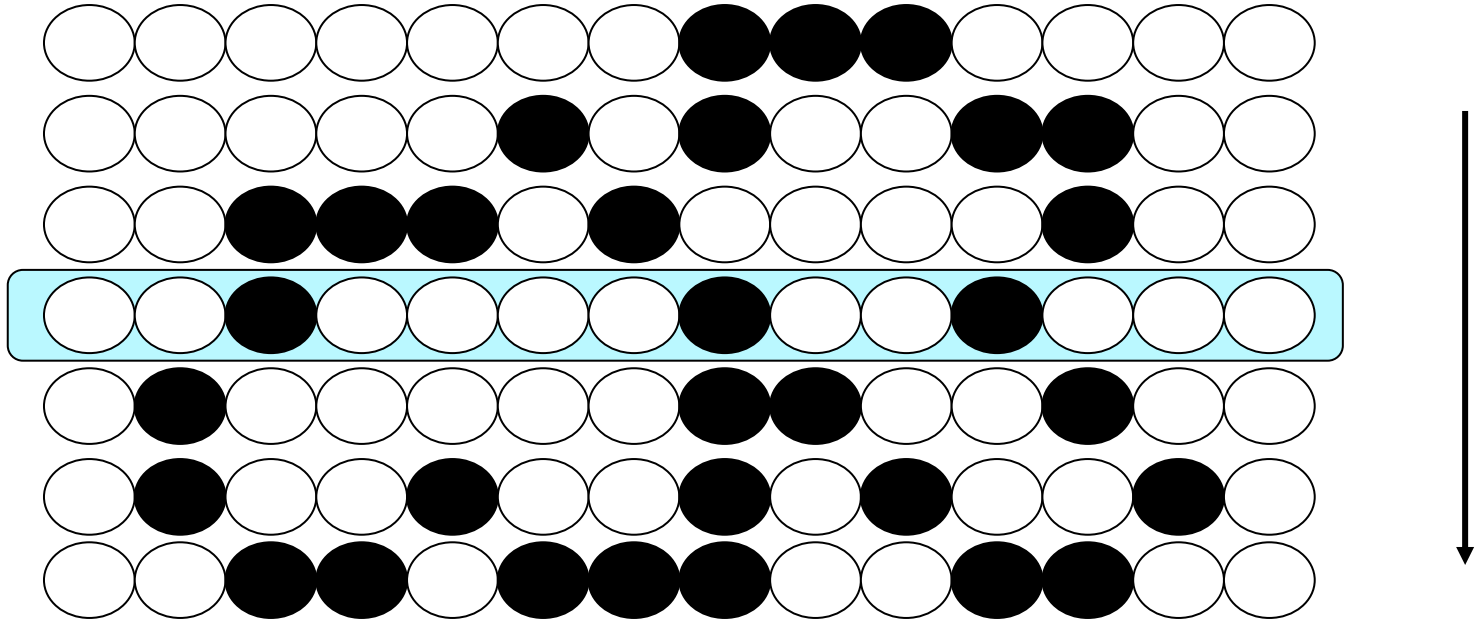
Aktuelles Element: - Nächste Queue-Elemente: -



Schauen wir uns jetzt das **Scan-Line-Verfahren** an

Idee des Scan-Line-Verfahrens

Von oben nach unten wird eine waagerechte **Scan-Linie** analysiert. Wir identifizieren die Abschnitte, die innerhalb eines Polygons liegen und füllen diese entsprechend, dann gehen wir zur nächsten Scan-Line.





Aber woher wissen wir, was **innen** und **außen** in Bezug auf ein Polygon bedeutet?

Scan-Line-Verfahren (erste Version)

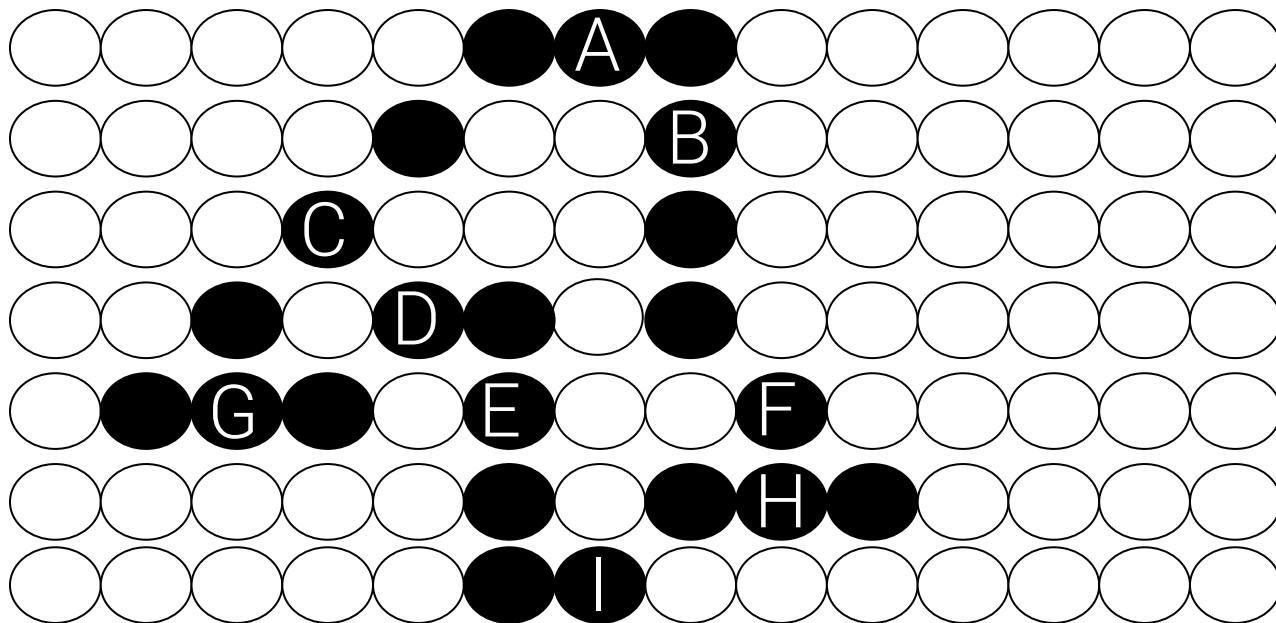
Folgende Schritte werden zum Füllen eines Polygons abgearbeitet. Dabei liegen die Polygonkanten A, B, C, D, E , ... usw. mit ihren Start- und Endpunkten zur Verfügung.

So verläuft beispielsweise die Kante A von Punkt $A1$ nach $A2$, mit $A1 = (x1, y1)$ und $A2 = (x2, y2)$.

- 1) **Sortiere alle Kanten** absteigend nach dem größten **y-Wert** innerhalb vom jeweiligen Start- und Endpunkt. Horizontale Kanten werden nicht einsortiert, diese werden direkt als Linie gezeichnet.
- 2) **Bewege die Scan-Linie** vom größten y -Wert nach unten in Richtung kleinstem y -Wert.
 - i) Ermittle die Kanten K , die die Scan-Linie schneiden.
 - ii) **Sortiere** alle Kanten K aufsteigend nach dem **x-Wert** der Schnittpunkte.
 - iii) Alle Punkte der Scan-Linie, die einer ungeraden Anzahl von Schnittpunkten folgen, liegen innerhalb des Polygons und werden eingefärbt.

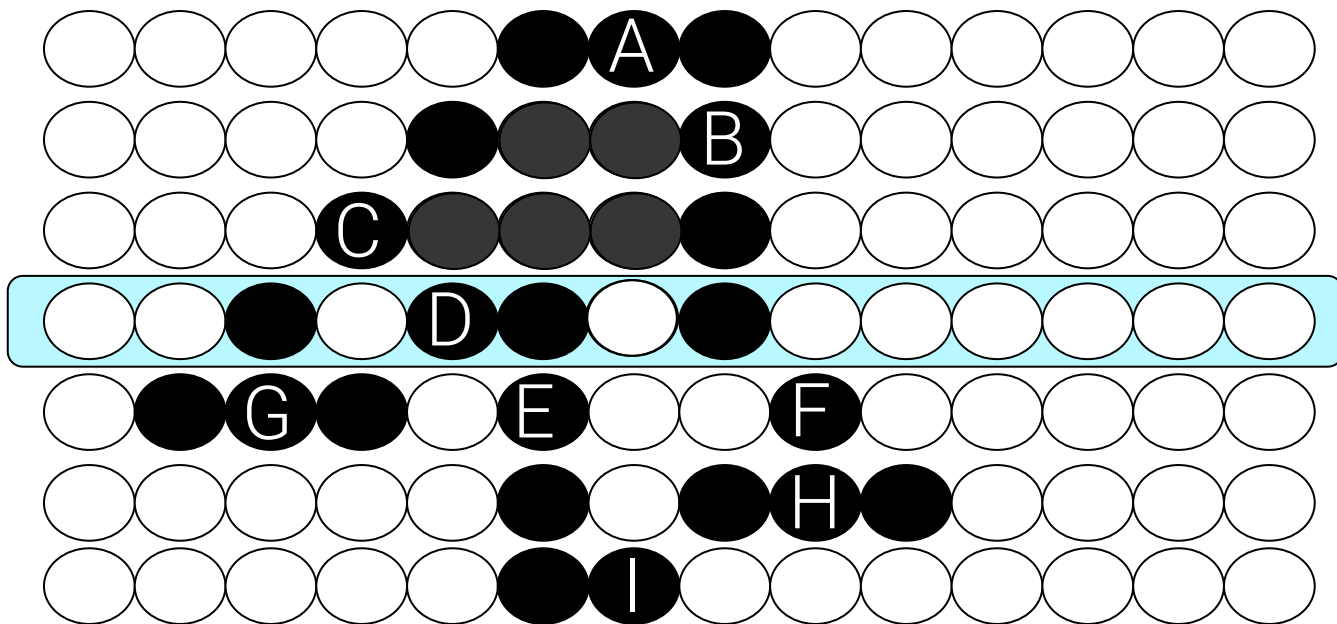
Beispiel zum Scan-Line-Verfahren

Gegeben sei das folgende Beispiel. Die Kanten wurden entsprechend ihrer Sortierung nach (1) benannt. Schauen wir uns eine konkrete Situation im Verlauf an.

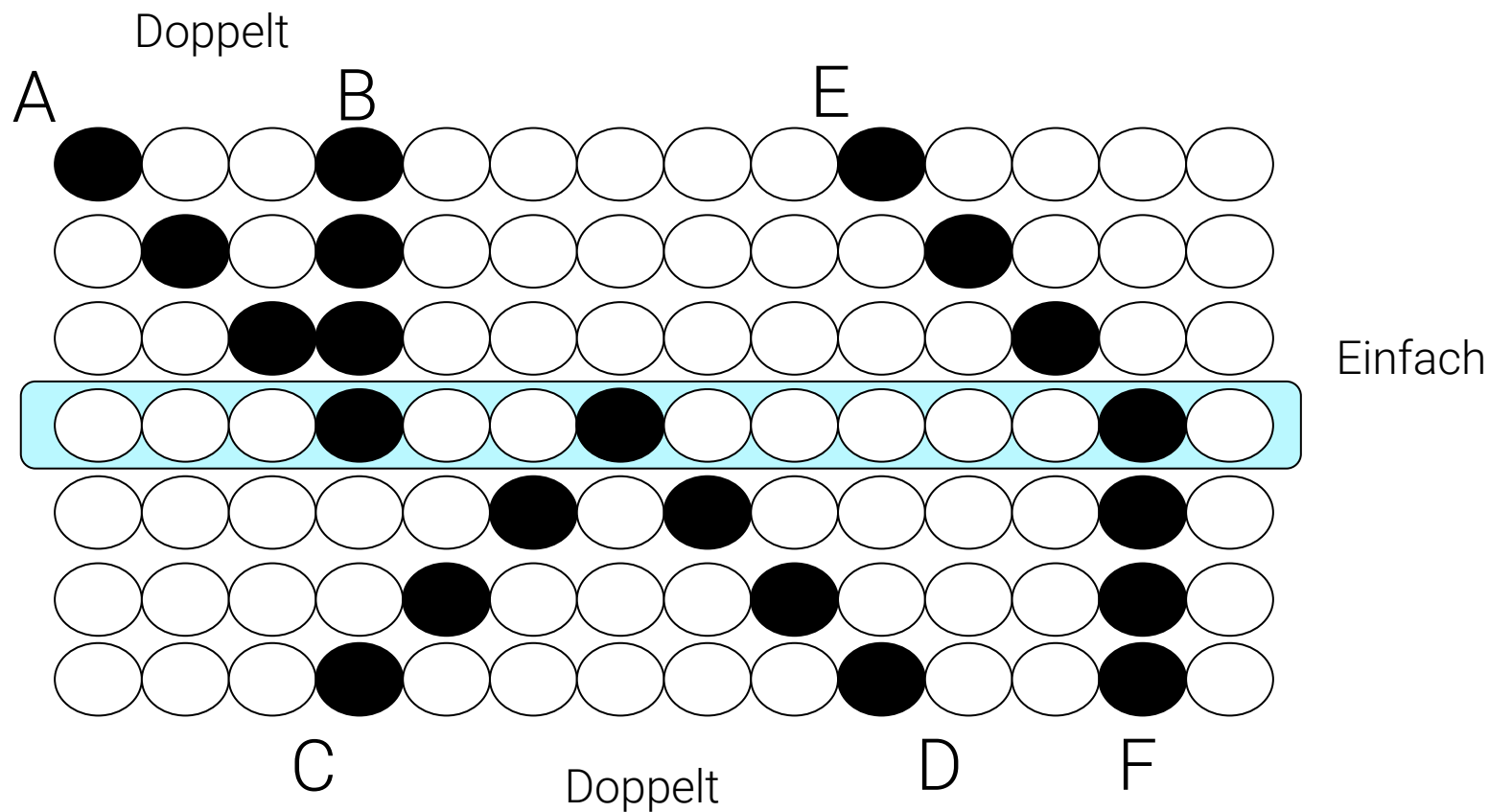


Beispiel zum Scan-Line-Verfahren

Gegeben sei das folgende Beispiel. Die Kanten wurden entsprechend ihrer Sortierung nach (1) benannt. Schauen wir uns eine konkrete Situation im Verlauf an.



Betrachtung der Spezialfälle



Scan-Line-Verfahren (erste Version)

Folgende Schritte werden zum Füllen eines Polygons abgearbeitet. Dabei liegen die Polygonkanten $A, B, C, D, E, \dots$ usw. mit ihren Start- und Endpunkten zur Verfügung.

So verläuft beispielsweise die Kante A von Punkt $A1$ nach $A2$, mit $A1 = (x1, y1)$ und $A2 = (x2, y2)$.

- 1) Sortiere alle Kanten absteigend nach dem größten y -Wert innerhalb vom jeweiligen Start- und Endpunkt. Horizontale Kanten werden nicht einsortiert, diese werden direkt als Linie gezeichnet.
- 2) Bewege die Scan-Linie vom größten y -Wert nach unten in Richtung kleinstem y -Wert.
 - i) Ermittle die Kanten K , die die Scan-Linie schneiden.
 - ii) **Sortiere alle Kanten K aufsteigend nach dem x -Wert der Schnittpunkte.**
 - iii) Alle Punkte der Scan-Linie, die einer ungeraden Anzahl von Schnittpunkten folgen, liegen innerhalb des Polygons und werden eingefärbt.



Jetzt verbessern wir die vorgestellte Version

Scan-Line-Verfahren (verbesserte Version)

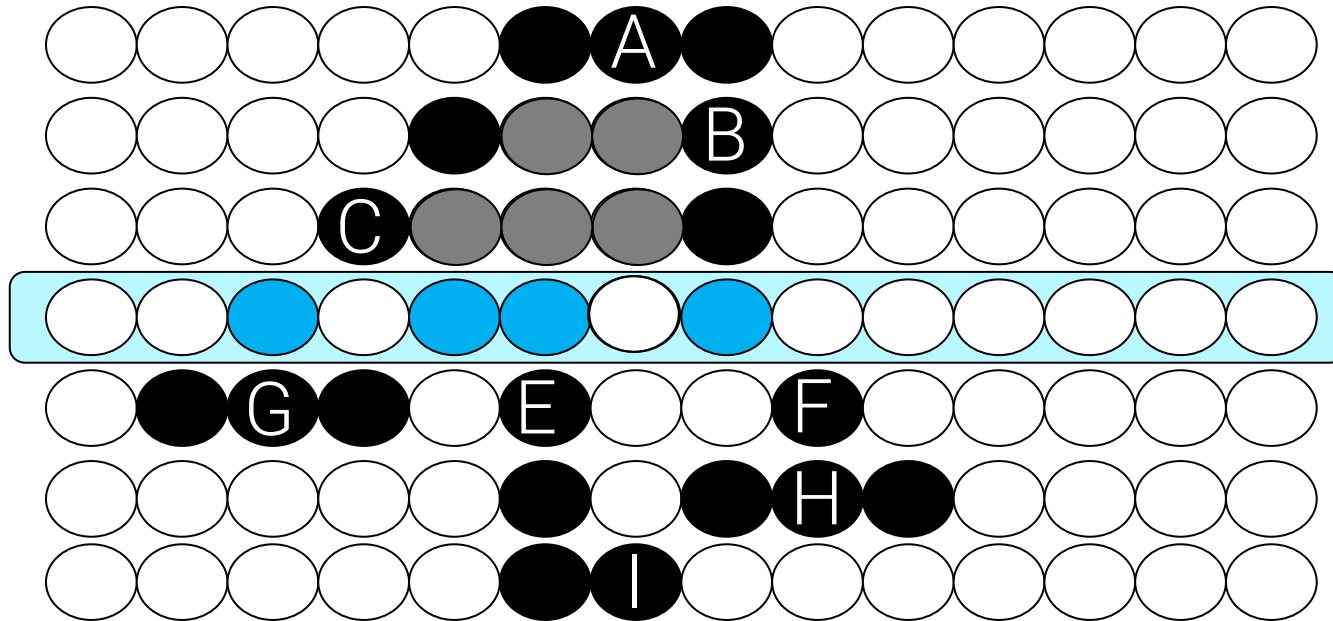
Folgende Schritte werden zum Füllen eines Polygons abgearbeitet. Dabei liegen die Polygonkanten A, B, C, D, E , ... usw. mit ihren Start- und Endpunkten zur Verfügung.

So verläuft beispielsweise die Kante A von Punkt $A1$ nach $A2$, mit $A1 = (x1, y1)$ und $A2 = (x2, y2)$.

- 1) Sortiere alle Kanten absteigend nach dem größten y -Wert innerhalb vom jeweiligen Start- und Endpunkt. Horizontale Kanten werden nicht einsortiert, diese werden direkt als Linie gezeichnet.
- 2) Bewege die Scan-Linie vom größten y -Wert nach unten in Richtung kleinstem y -Wert.
 - i) Ermittle die Kanten K , die die Scan-Linie schneiden.
 - ii) Sortiere alle Kanten K aufsteigend nach dem x -Wert der Schnittpunkte.
Liegen die beiden Kanten eines Schnittpunkts oberhalb oder unterhalb, so zählt der Schnittpunkt doppelt. Liegen die Kanten oberhalb und unterhalb, so zählt dieser Schnittpunkt nur einfach.
 - iii) Alle Punkte der Scan-Linie, die einer ungeraden Anzahl von Schnittpunkten folgen, liegen innerhalb des Polygons und werden eingefärbt.

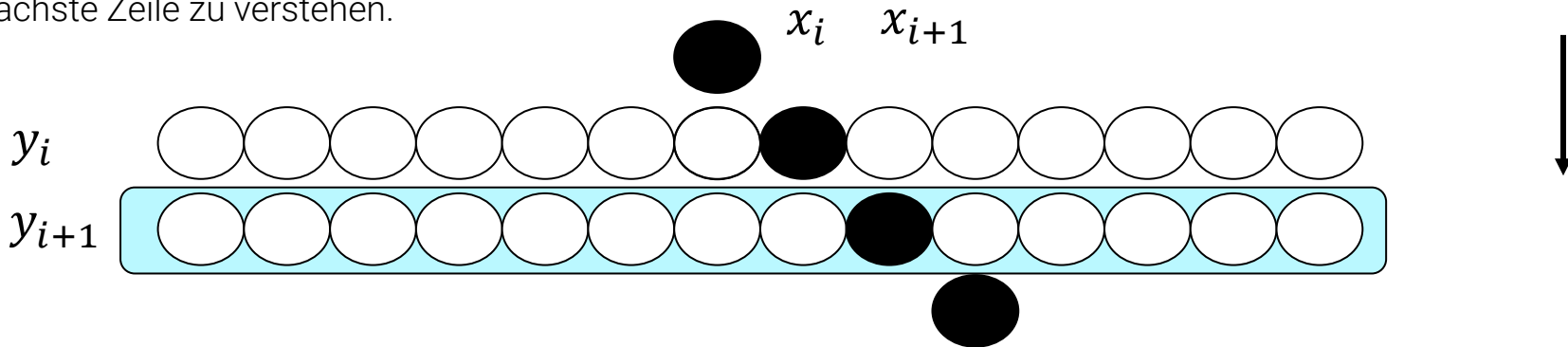
Schnittpunkte vorhersagen

Es gibt eine einfache Möglichkeit, die zukünftigen Schnittpunkte bereits schneidender Kanten zu ermitteln. Wir isolieren mal eine Beispielkante, um die Vorhersage des nächsten Schnittpunktes beim Übergang der Scan-Linie in die nächste Zeile zu verstehen.



Schnittpunkte vorhersagen

Es gibt eine einfache Möglichkeit, die zukünftigen Schnittpunkte bereits schneidender Kanten zu ermitteln. Wir isolieren mal eine Beispielkante, um die Vorhersage des nächsten Schnittpunktes beim Übergang der Scan-Linie in die nächste Zeile zu verstehen.



$$m = \frac{y_i - y_{i+1}}{x_i - x_{i+1}}$$

$y_i - y_{i+1}$ kennen wir bereits, da die Scan-Linie immer um 1 nach unten wandert.

Der neue Schnittpunkt ergibt sich dann bei $S_{neu} := (x_i - \frac{1}{m}, y_{i+1})$

$$f(x) = y = m \cdot x + b$$

$$m = \frac{dy}{dx} \quad \begin{array}{l} dx = qx - px \\ dy = qy - py \end{array}$$

Implementierungsbeispiel – Scan-Line-Verfahren

Schauen wir uns hier das Beispiel in einer interaktiven Umgebung an. Es kann ein Polygon definiert werden.

Danach können wir das Scan-Line-Verfahren schrittweise durchführen.

Polygon: Klicke Gitterpunkte. Schließen durch Klick auf P0 (mind. 3 Punkte).
SPACE = nächste Scanline, B = zurück, A = Autoplay, R = Reset
Nach dem Schließen: Punkte ziehen möglich; Algorithmus wird dabei sauber zurückgesetzt.
Scanline-Regel: Kante aktiv genau dann, wenn $y \geq \min Y$ und $y < \max Y$.

Modus: Polygon zeichnen
aktuelle Zeile: -
Schritte: 0
Scanbereich: -



Legende:

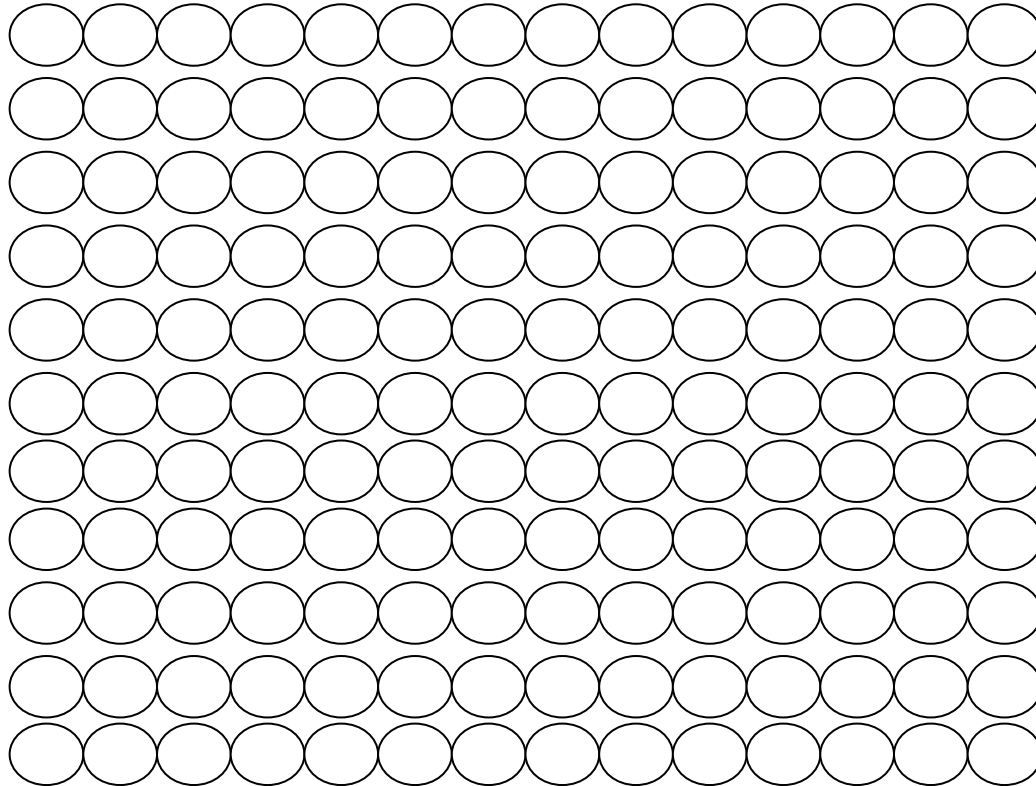
 bereits gefüllte Scanlines	 aktuelle Füllspanne	 Schnittpunkt	 aktuelle Scanline
--	--	--	---



Schauen wir uns jetzt das „Scan-Line-Verfahren“
für einen **Kreis** an

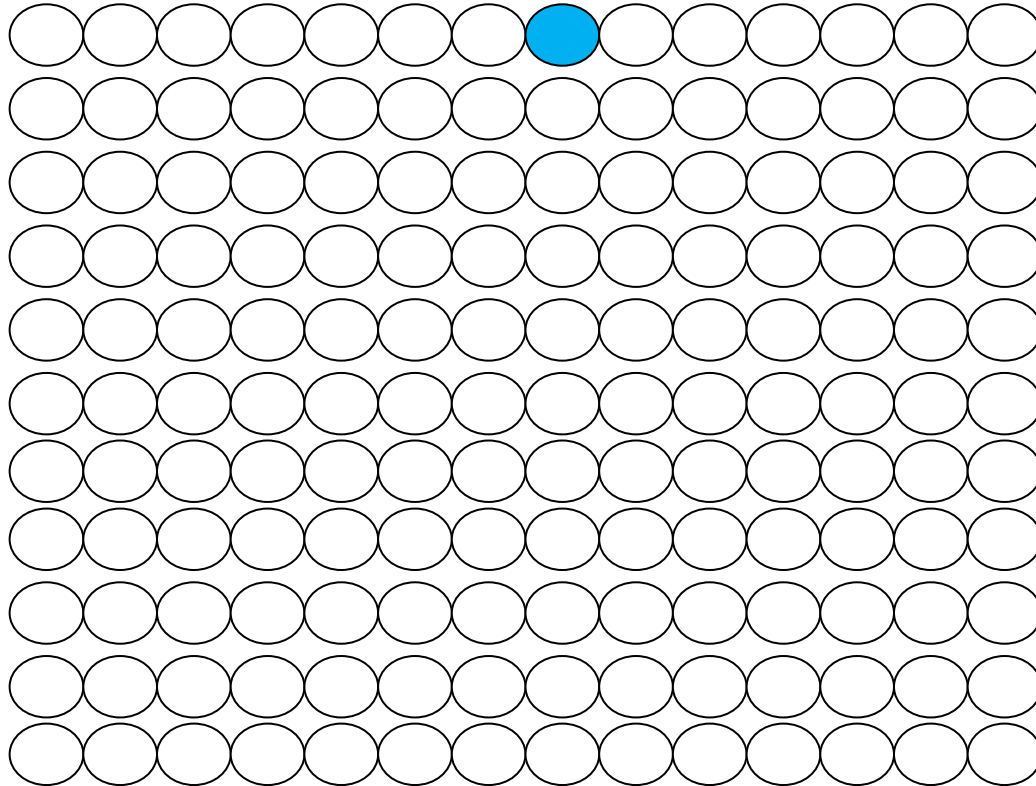
Kreisfüllen mit Scan-Line-Verfahren

Wir können das Kreisfüllen direkt mit dem Bresenham-Algorithmus zum Kreiszeichnen verbinden:



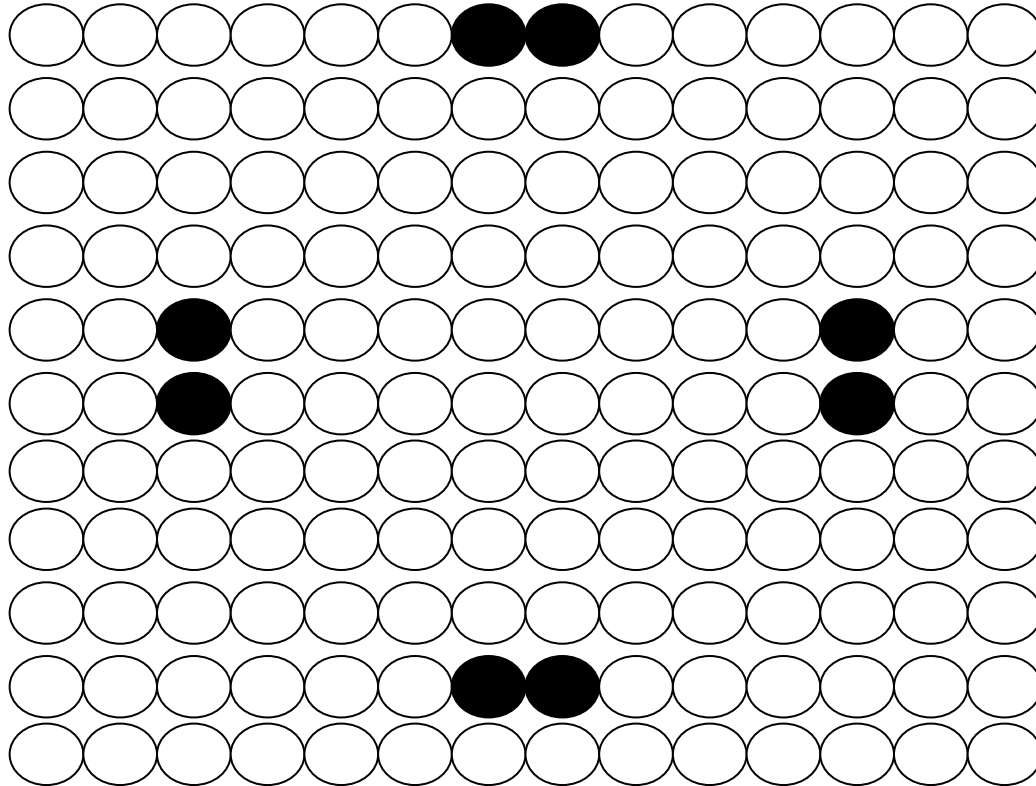
Kreisfüllen mit Scan-Line-Verfahren

Wir können das Kreisfüllen direkt mit dem Bresenham-Algorithmus zum Kreiszeichnen verbinden:



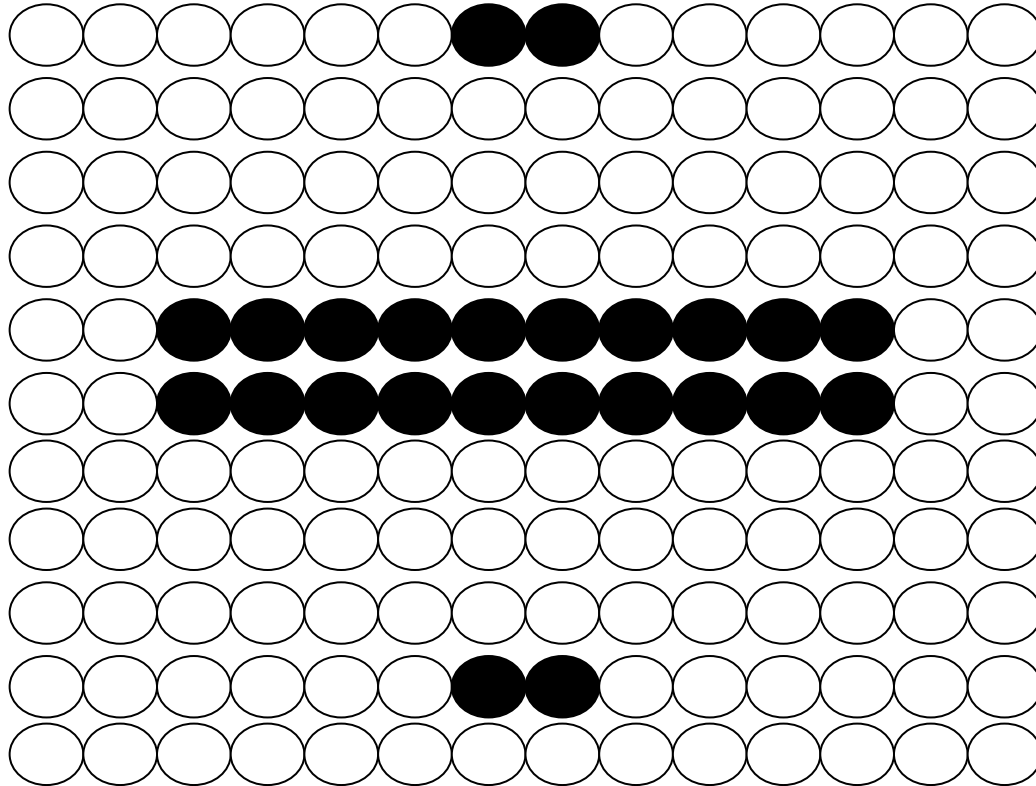
Kreisfüllen mit Scan-Line-Verfahren

Wir können das Kreisfüllen direkt mit dem Bresenham-Algorithmus zum Kreiszeichnen verbinden:



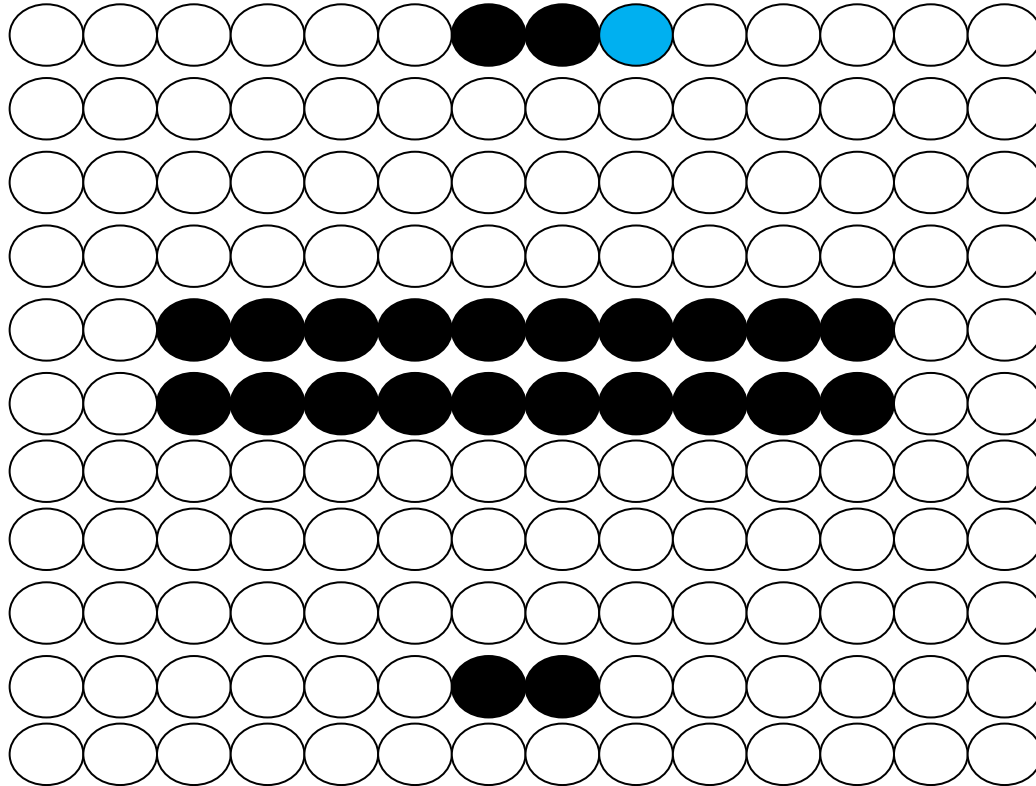
Kreisfüllen mit Scan-Line-Verfahren

Wir können das Kreisfüllen direkt mit dem Bresenham-Algorithmus zum Kreiszeichnen verbinden:



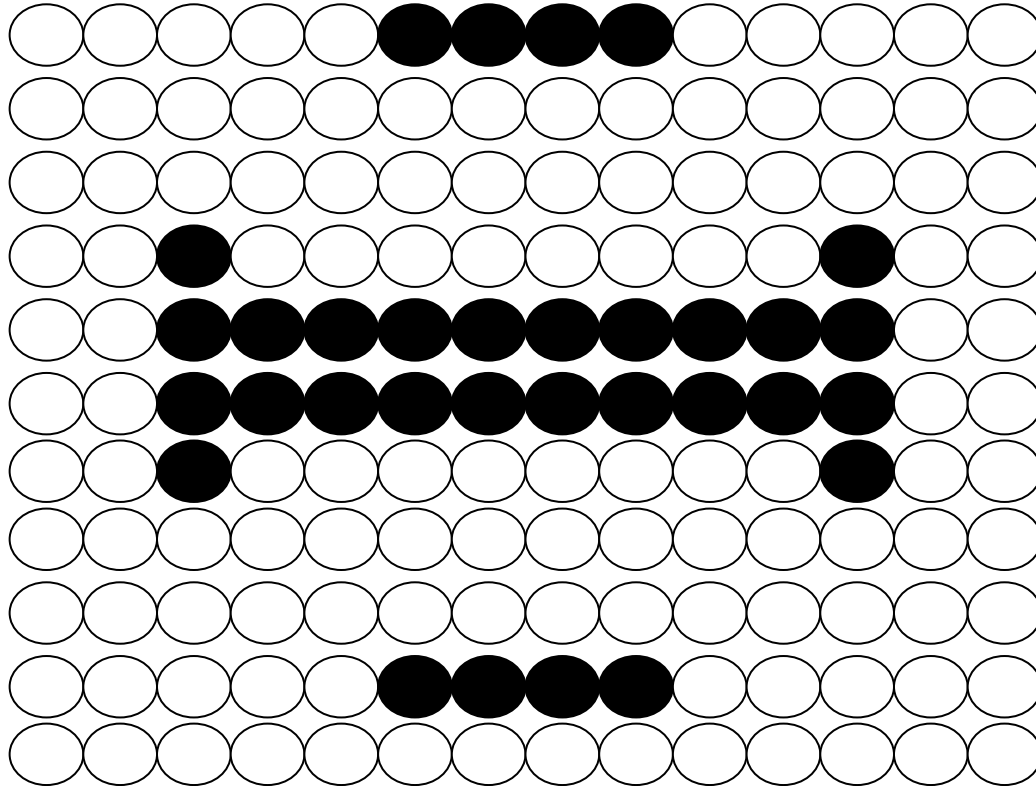
Kreisfüllen mit Scan-Line-Verfahren

Wir können das Kreisfüllen direkt mit dem Bresenham-Algorithmus zum Kreiszeichnen verbinden:



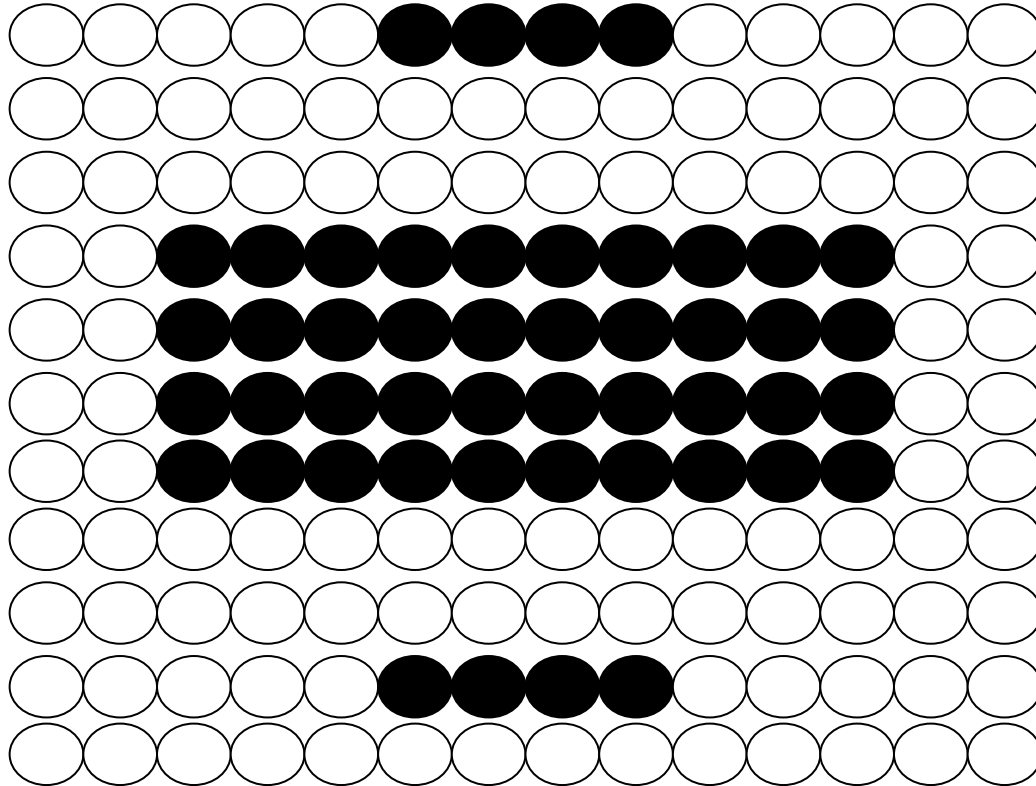
Kreisfüllen mit Scan-Line-Verfahren

Wir können das Kreisfüllen direkt mit dem Bresenham-Algorithmus zum Kreiszeichnen verbinden:



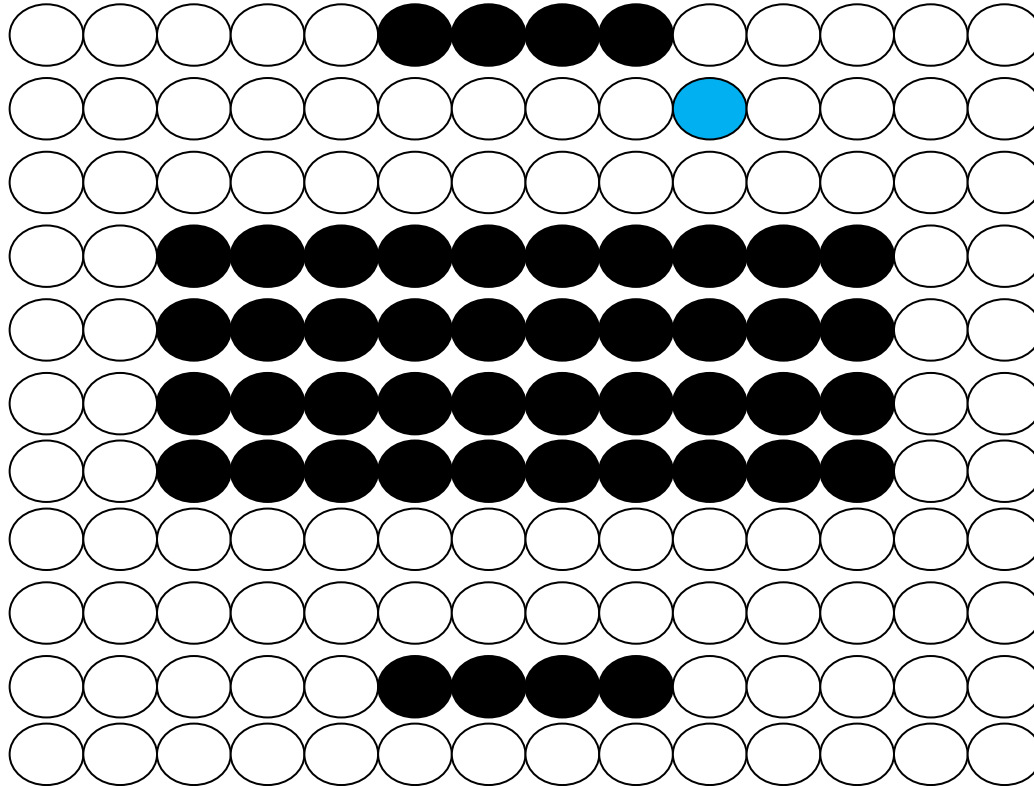
Kreisfüllen mit Scan-Line-Verfahren

Wir können das Kreisfüllen direkt mit dem Bresenham-Algorithmus zum Kreiszeichnen verbinden:



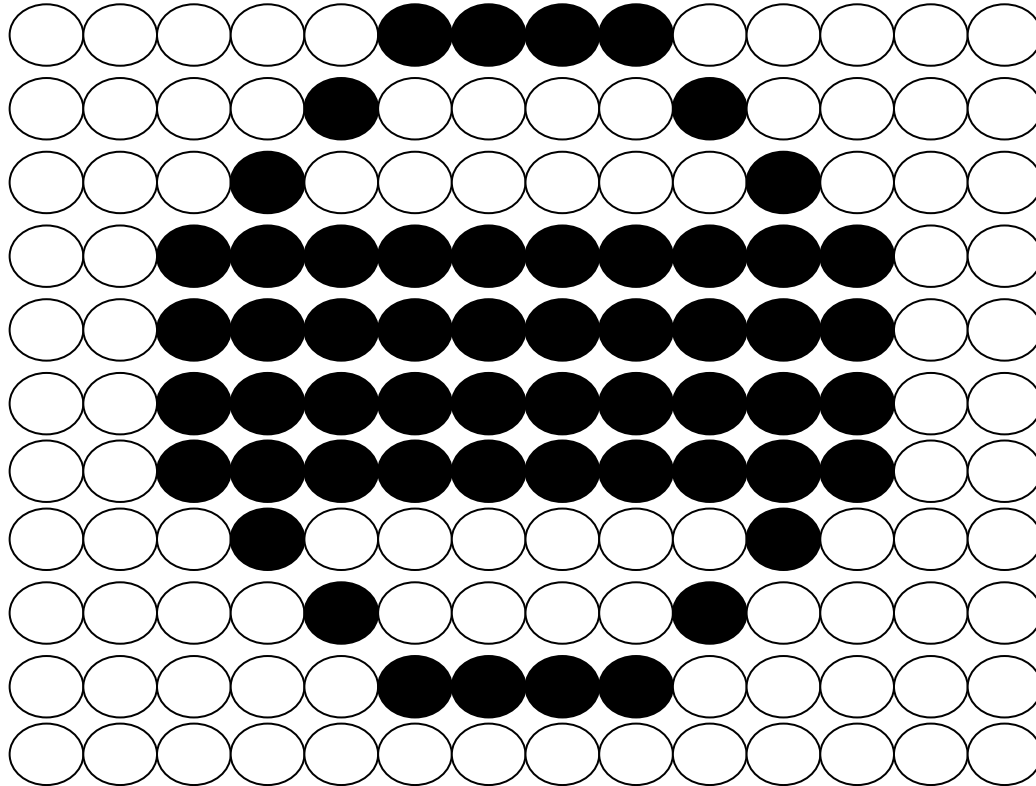
Kreisfüllen mit Scan-Line-Verfahren

Wir können das Kreisfüllen direkt mit dem Bresenham-Algorithmus zum Kreiszeichnen verbinden:



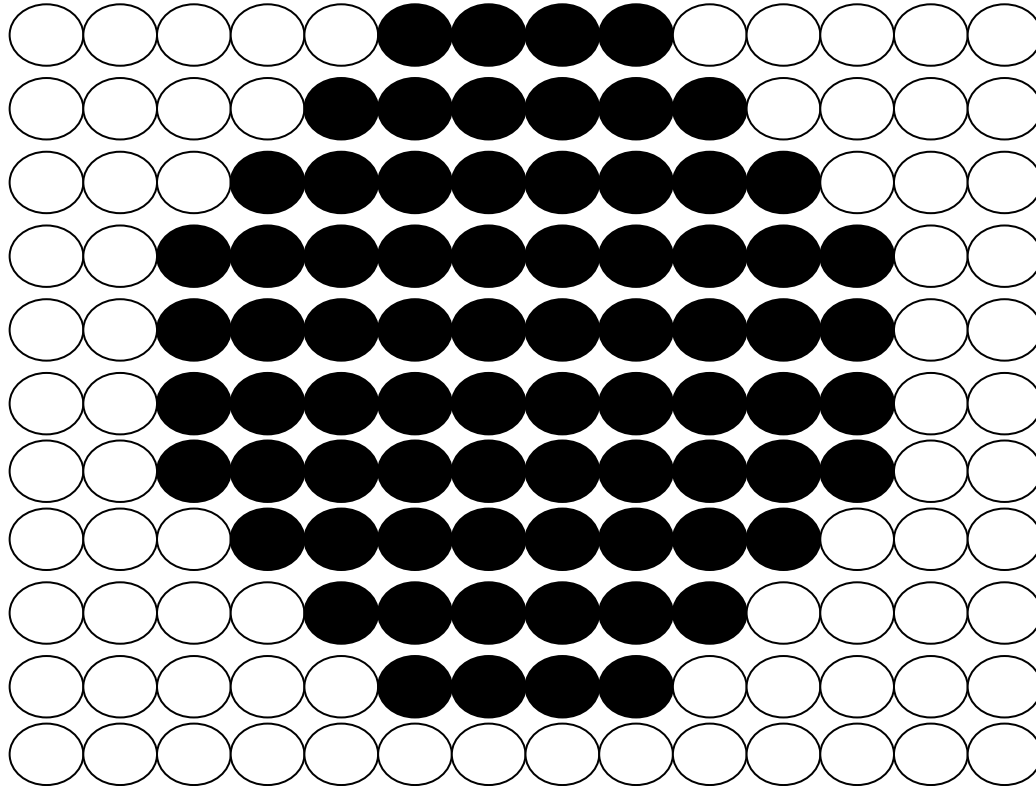
Kreisfüllen mit Scan-Line-Verfahren

Wir können das Kreisfüllen direkt mit dem Bresenham-Algorithmus zum Kreiszeichnen verbinden:



Kreisfüllen mit Scan-Line-Verfahren

Wir können das Kreisfüllen direkt mit dem Bresenham-Algorithmus zum Kreiszeichnen verbinden:

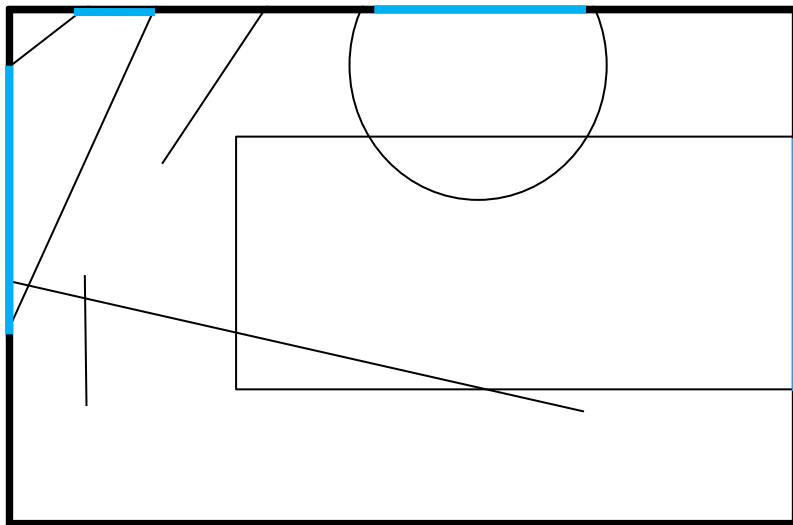




Wir können jetzt Polygone zeichnen und ausfüllen, aber wir sehen diese nicht immer vollständig?

Motivation zum Clipping

Wir haben es bei der Darstellung einer virtuellen Szene meistens nur mit einem kleinen Teilausschnitt zu tun. Idee: Komplexe Geometrien können sicherlich vor der Darstellung vereinfacht werden, um den Berechnungsaufwand für die Darstellung zu reduzieren.

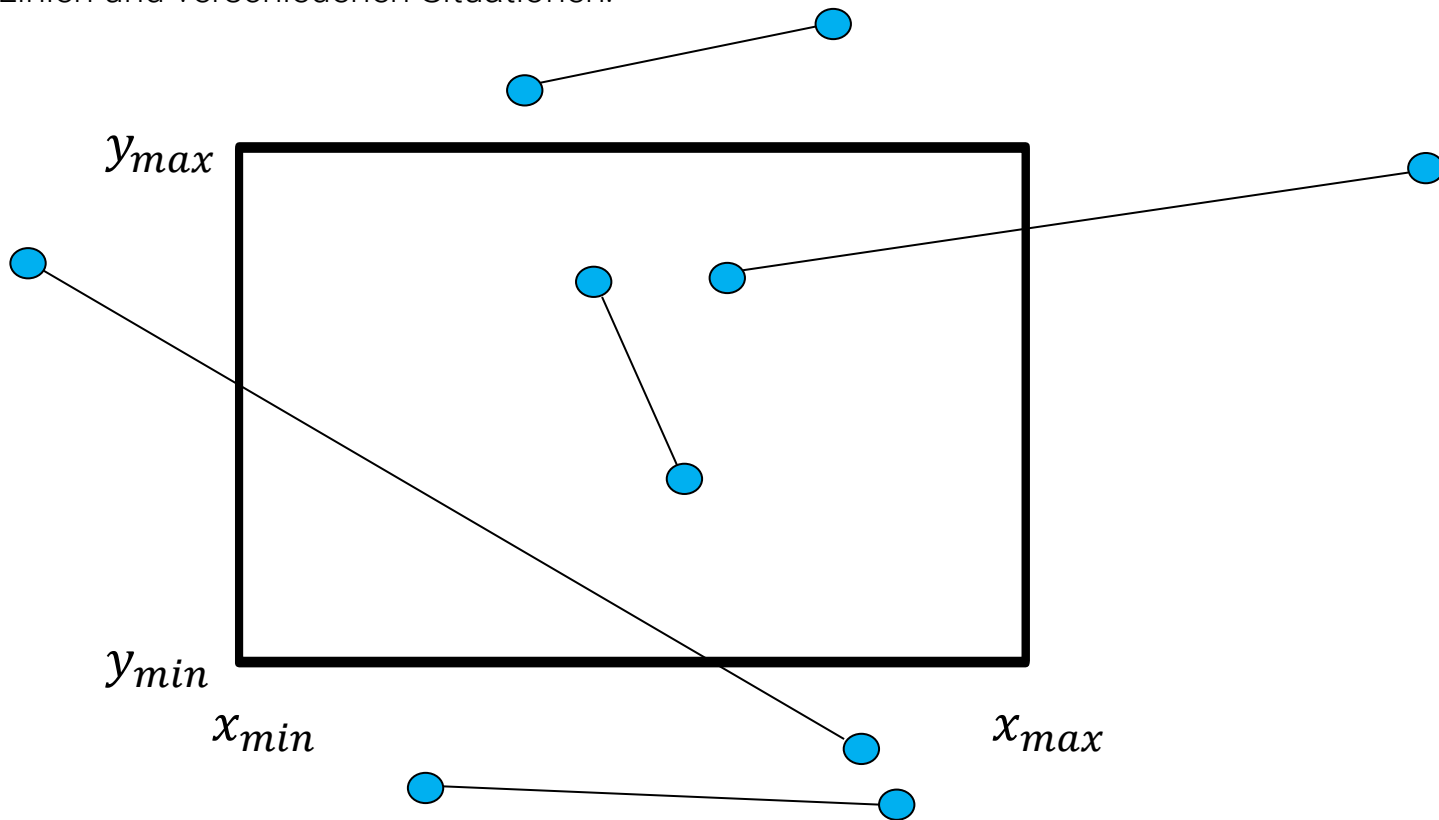


komplexe Szene

Clipping-Fenster

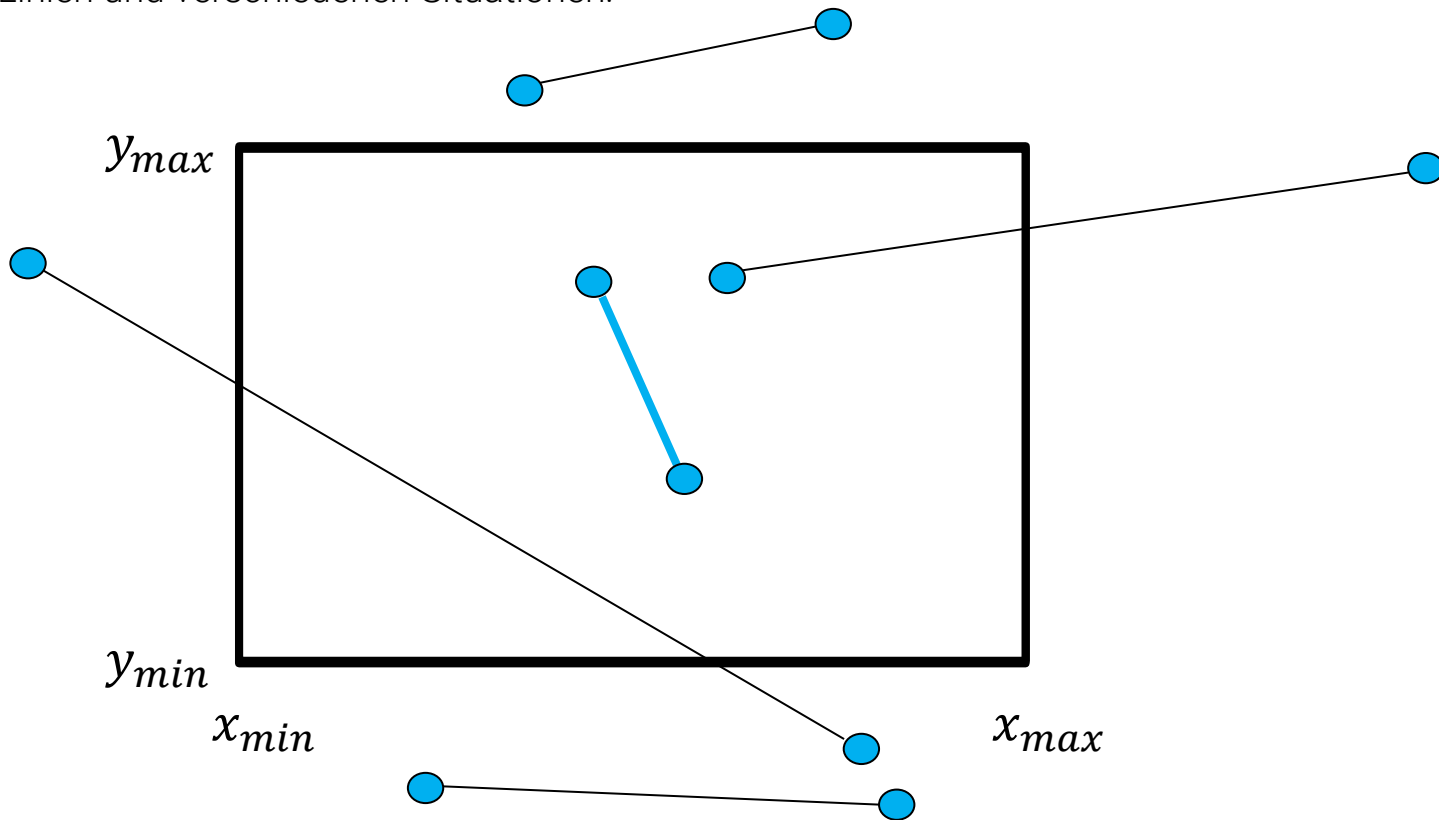
Clipping von Linien

Beispiel mit fünf Linien und verschiedenen Situationen:



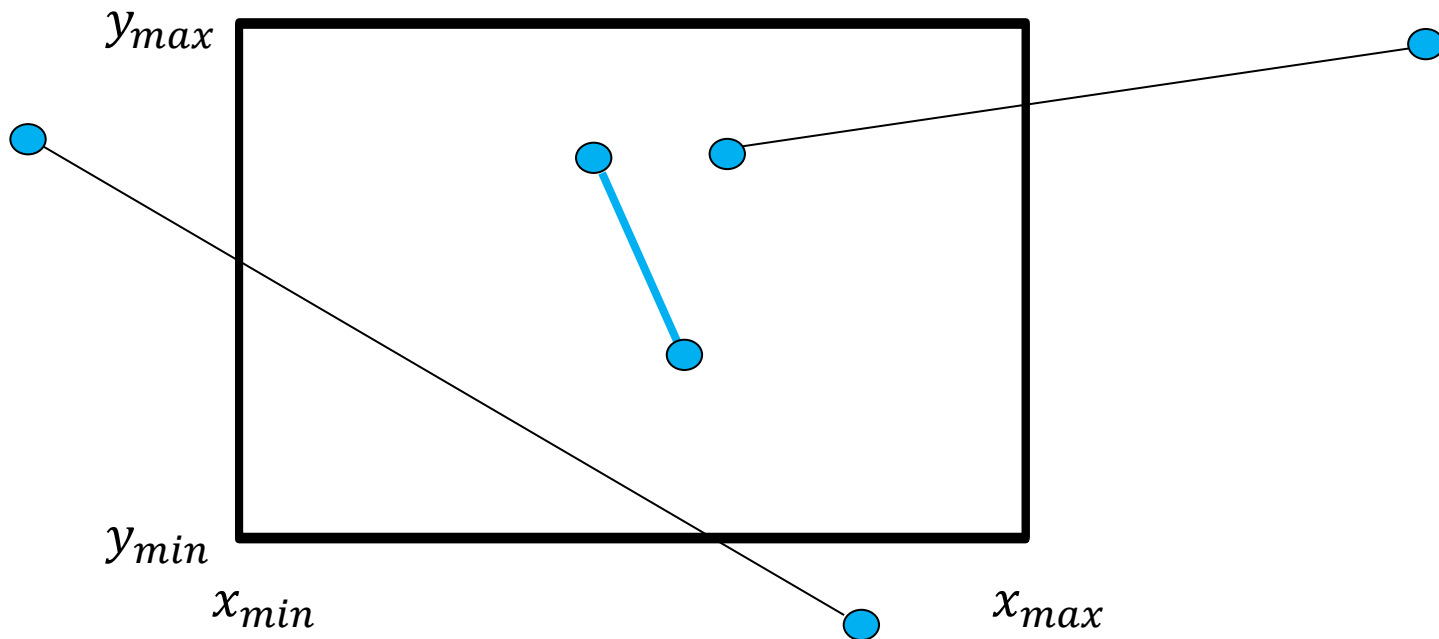
Clipping von Linien

Beispiel mit fünf Linien und verschiedenen Situationen:



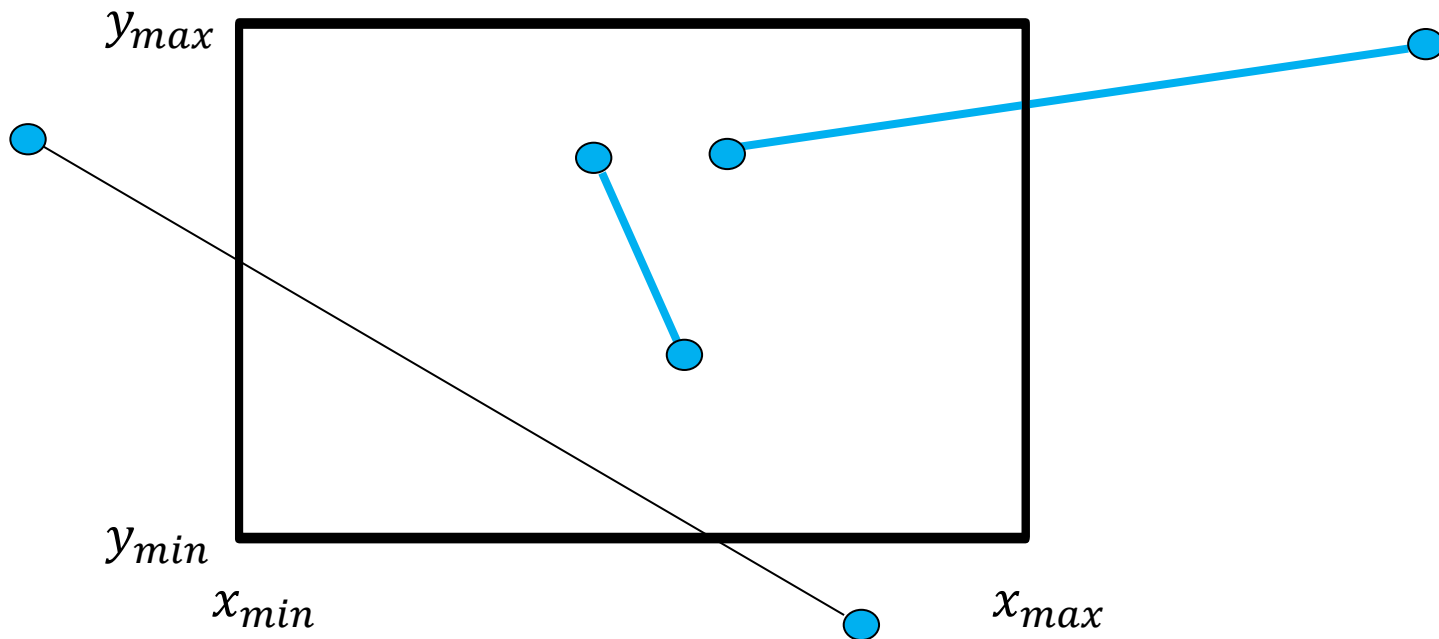
Clipping von Linien

Beispiel mit fünf Linien und verschiedenen Situationen:



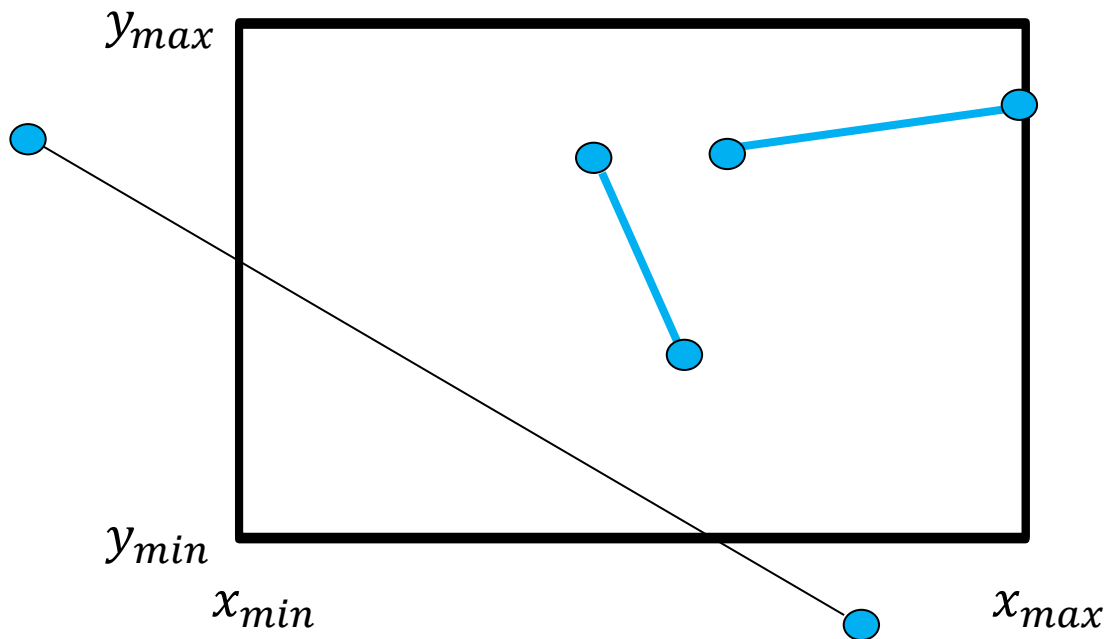
Clipping von Linien

Beispiel mit fünf Linien und verschiedenen Situationen:



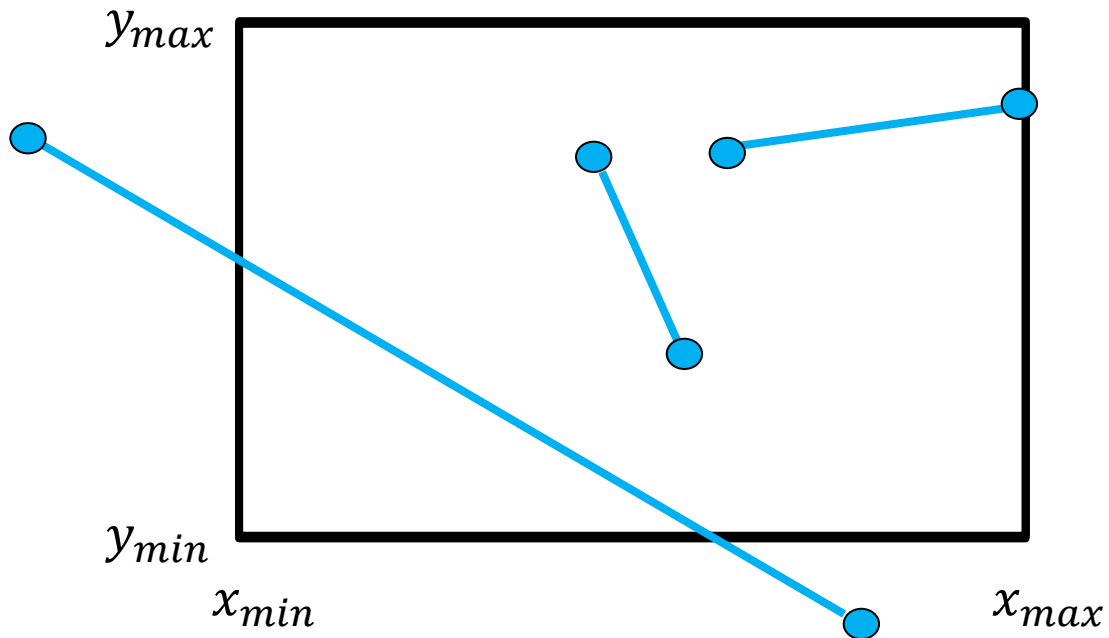
Clipping von Linien

Beispiel mit fünf Linien und verschiedenen Situationen:



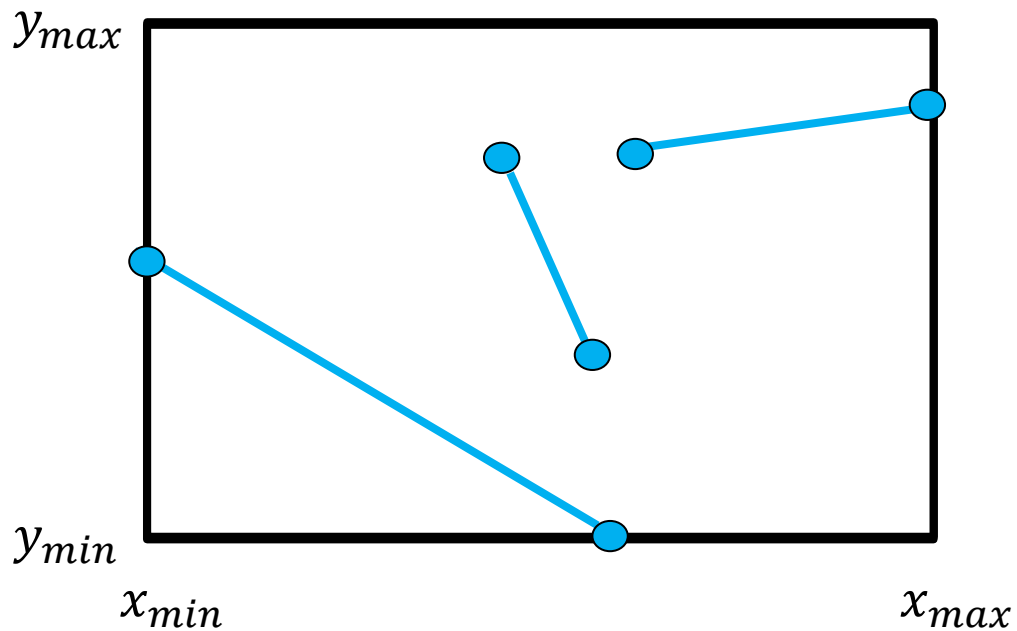
Clipping von Linien

Beispiel mit fünf Linien und verschiedenen Situationen:



Clipping von Linien

Beispiel mit fünf Linien und verschiedenen Situationen:

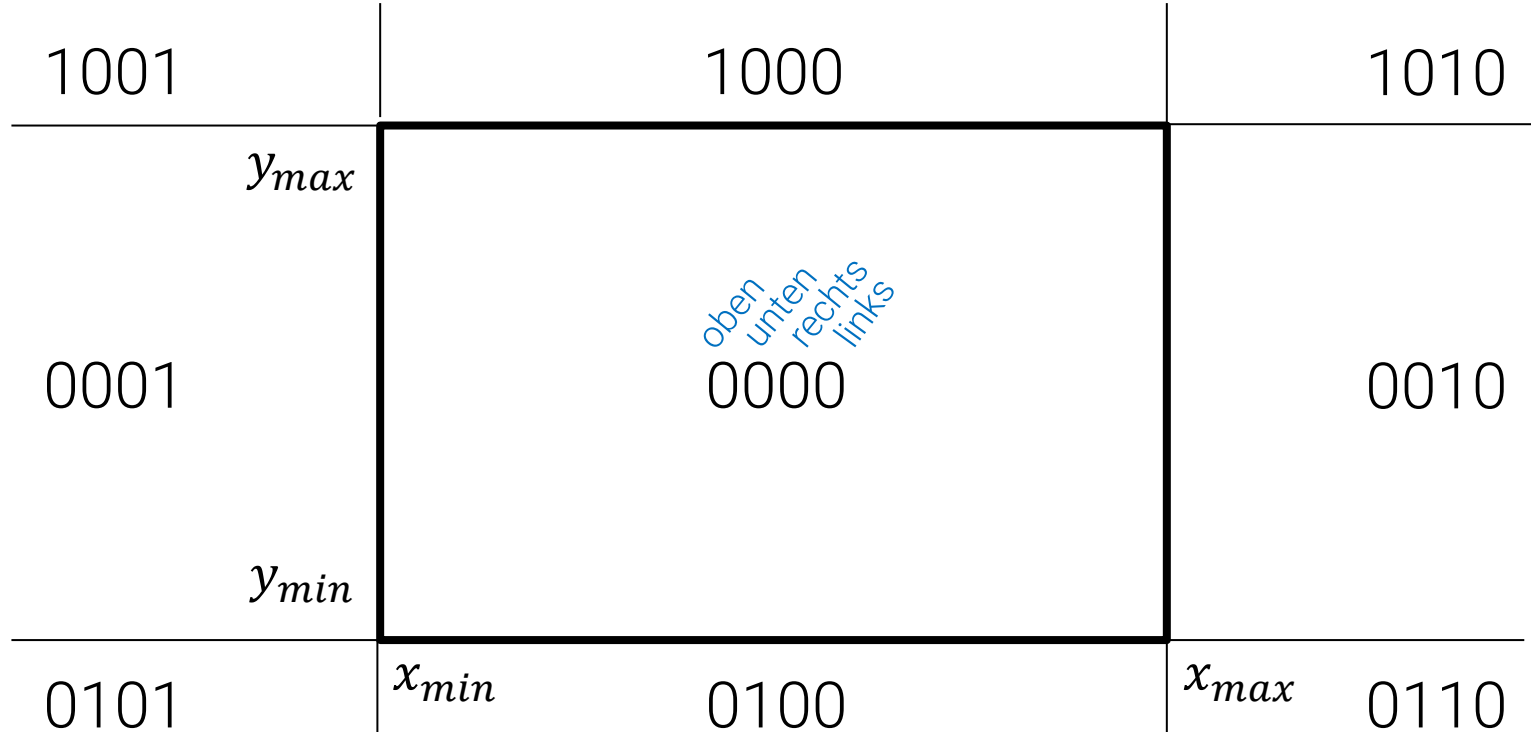




Wie gehen wir die Sache jetzt am besten an?

Idee von Cohen&Sutherland

Zunächst einmal codieren wir uns in geeigneter Form die Nachbarregionen um das Clipping-Fenster (der sogenannte [Region-Code](#)).



Region-Code ermitteln

Für die Berechnung des Codes können wir unser Wissen zum Thema [Binärcodierung](#) (bzw. Boolesche Algebra) gezielt einsetzen.

```
private final byte CENTER    = 0;    // 0000
private final byte LEFT     = 1;    // 0001
private final byte RIGHT    = 2;    // 0010
private final byte BOTTOM   = 4;    // 0100
private final byte TOP      = 8;    // 1000
```

Java

```
public static byte regionCode(int x, int y) {
    byte result = CENTER;
    if (x < xMin) result = LEFT;
    if (x > xMax) result = RIGHT;
    if (y < yMin) result |= BOTTOM;
    if (y > yMax) result |= TOP;
    return result;
}
```

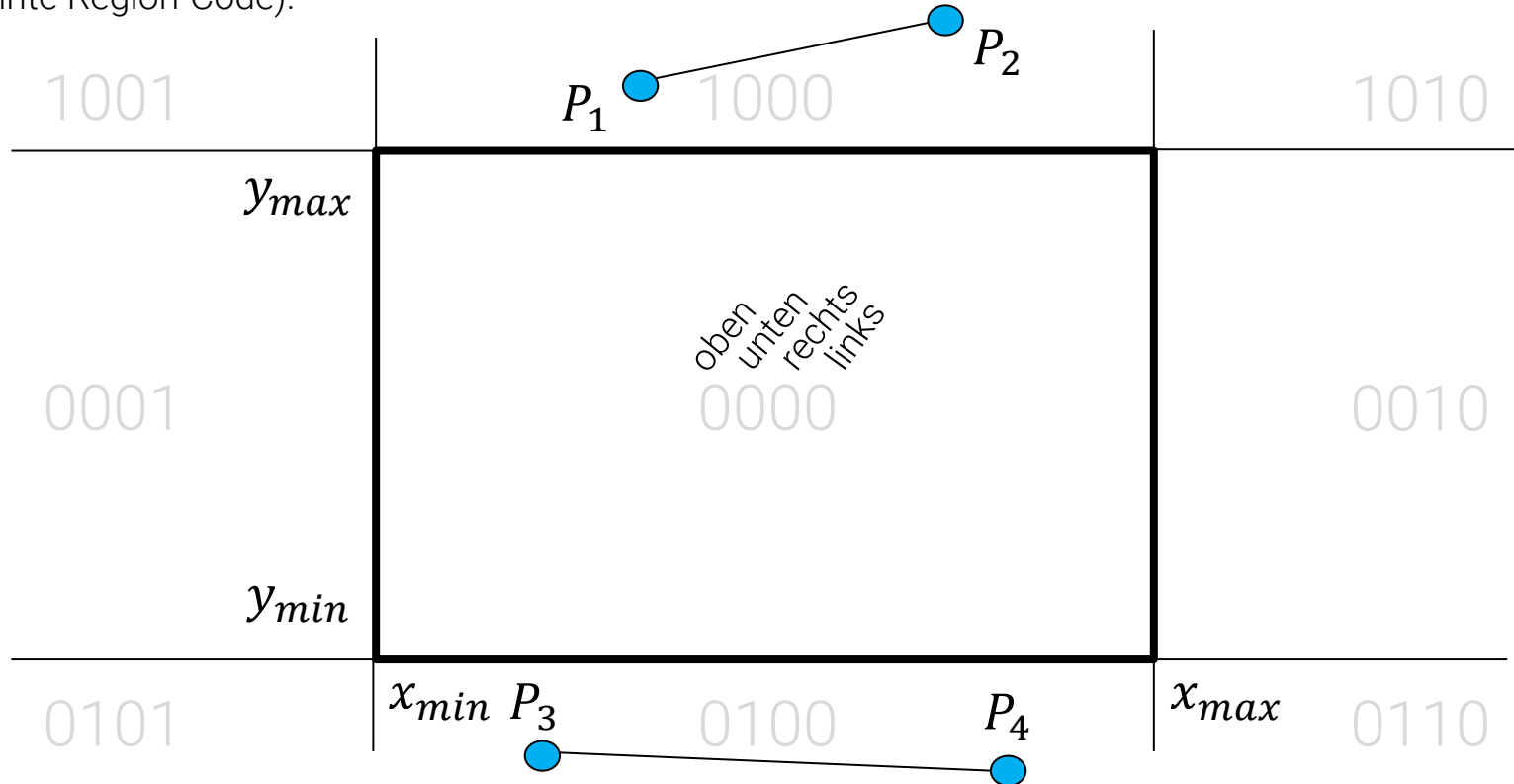




Was bringt uns hier der **Region-Code**?

Idee von Cohen&Sutherland

Zunächst einmal codieren wir uns in geeigneter Form die Nachbarregionen um das Clipping-Fenster (der sogenannte Region-Code).



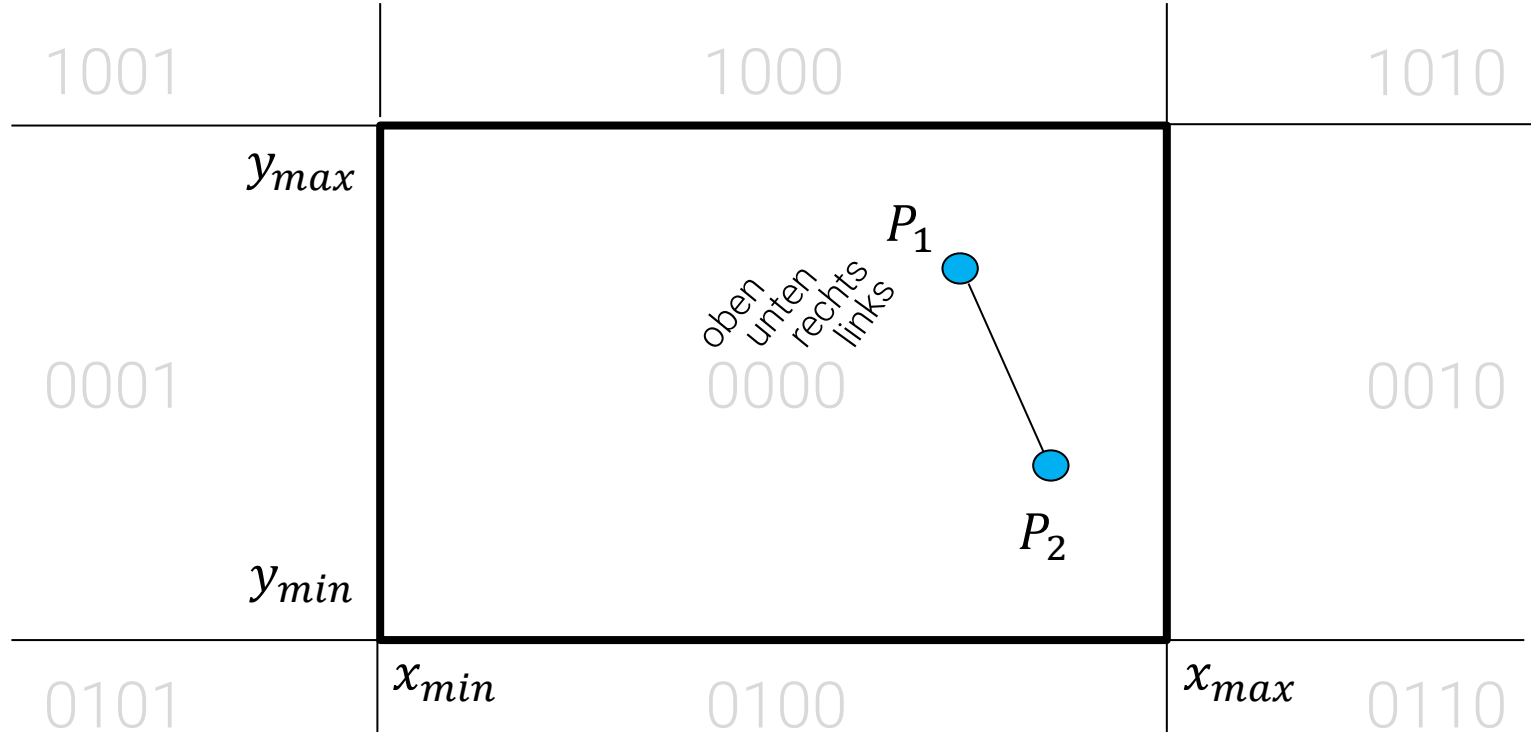
Region-Code nutzen

Fall 1: Linie liegt komplett außerhalb (bzgl. einer der Seiten, also komplett links, rechts, oben oder unten),
z. B. $rC(P1)=1000$, $rC(P2)=1000$ bzw. $rC(P3)=0100$, $rC(P4)=0100$.

wenn $rC(P1) \& rC(P2) \neq 0$, dann liegt die Linie **komplett außerhalb**

Idee von Cohen&Sutherland

Zunächst einmal codieren wir uns in geeigneter Form die Nachbarregionen um das Clipping-Fenster (der sogenannte Region-Code).



Region-Code nutzen

Fall 1: Linie liegt komplett außerhalb (bzgl. einer der Seiten, also komplett links, rechts, oben oder unten),
z. B. $rC(P1)=1000$, $rC(P2)=1000$ bzw. $rC(P3)=0100$, $rC(P4)=0100$.

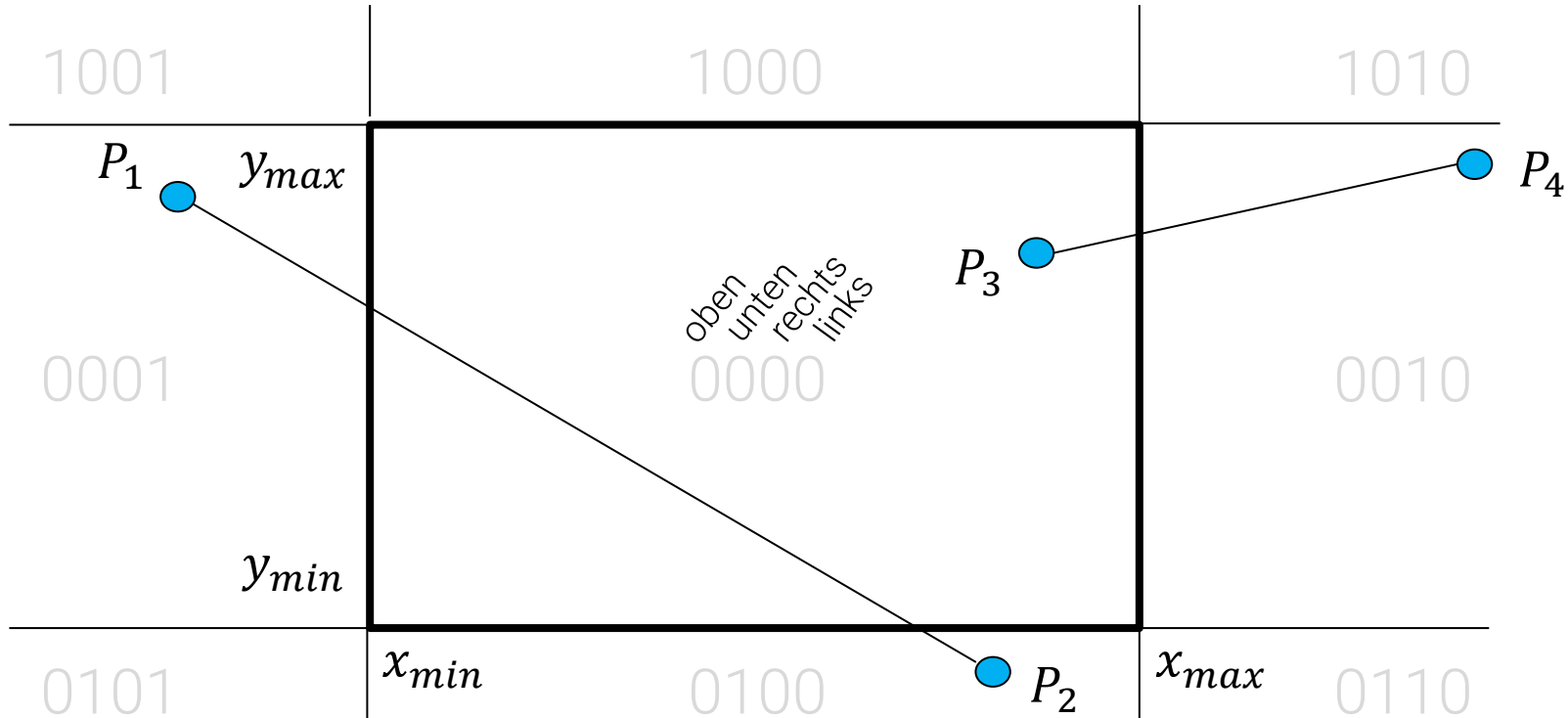
wenn $rC(P1) \& rC(P2) \neq 0$, dann liegt die Linie komplett außerhalb

Fall 2: Linie liegt komplett innerhalb, z. B. $rC(P1)=0000$, $rC(P2)=0000$.

wenn $rC(P1) | rC(P2) == 0$, dann liegt Linie **komplett innerhalb**

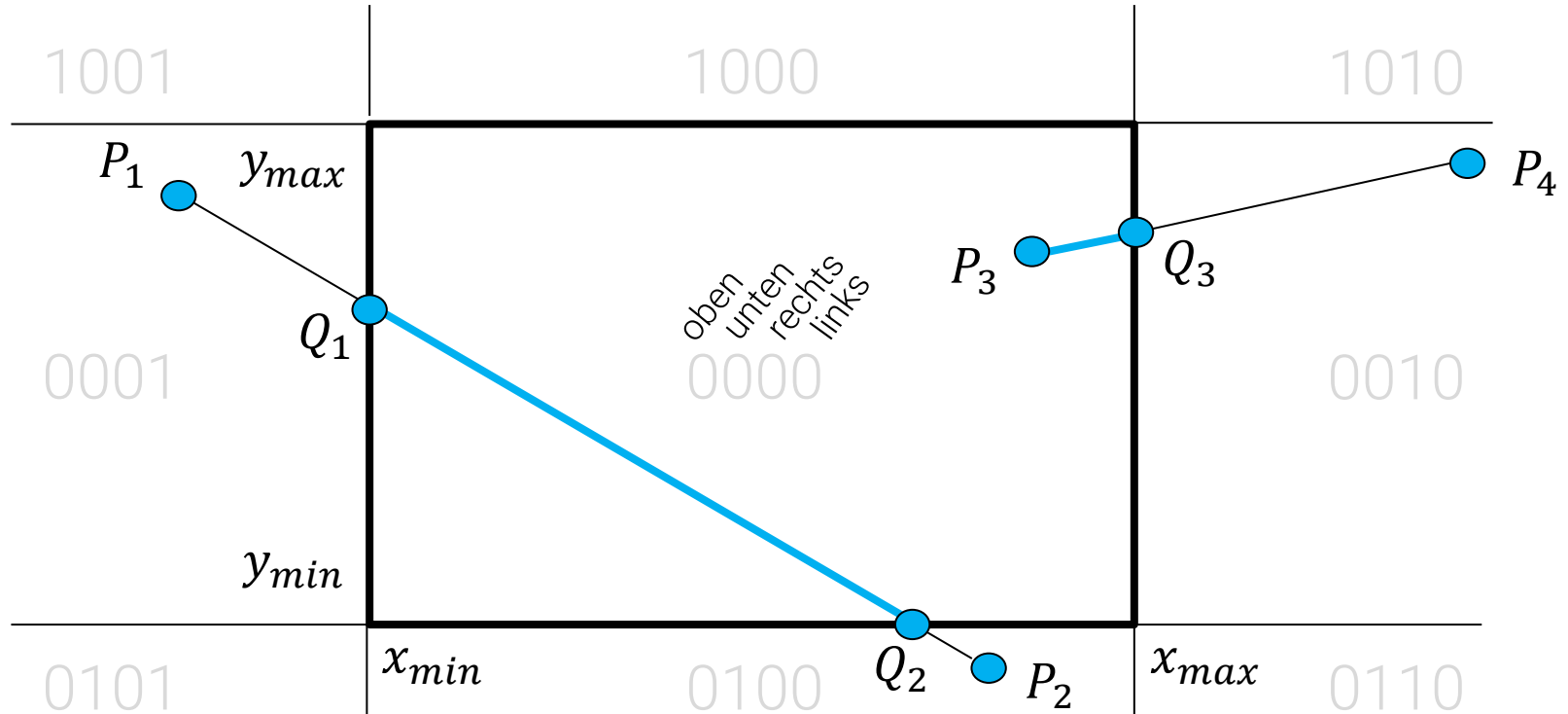
Idee von Cohen&Sutherland

Zunächst einmal codieren wir uns in geeigneter Form die Nachbarregionen um das Clipping-Fenster (der sogenannte Region-Code).



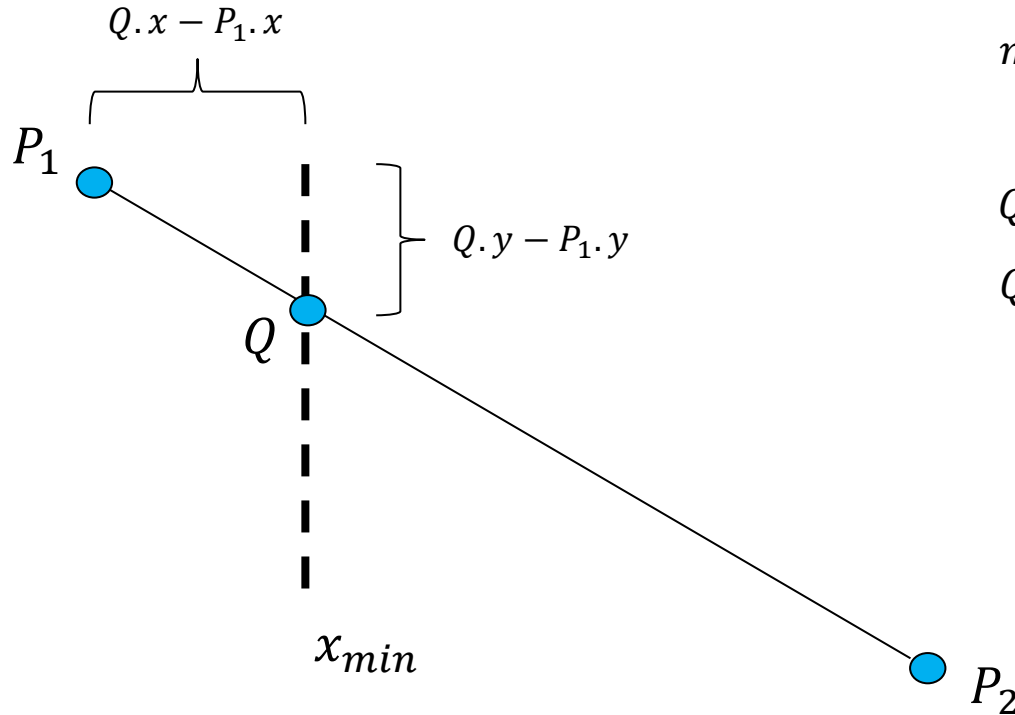
Idee von Cohen&Sutherland

Zunächst einmal codieren wir uns in geeigneter Form die Nachbarregionen um das Clipping-Fenster (der sogenannte Region-Code).



Schnittpunkt berechnen

Schauen wir uns exemplarisch die Berechnung eines Schnittpunktes an.



$$m = \frac{Q.y - P_1.y}{Q.x - P_1.x} = \frac{P_2.y - P_1.y}{P_2.x - P_1.x}$$

$$Q.x = x_{min}$$

$$Q.y = m \cdot (Q.x - P_1.x) + P_1.y$$

Region-Code nutzen

Fall 1: Linie liegt komplett außerhalb (bzgl. einer der Seiten, also komplett links, rechts, oben oder unten),
z. B. $rC(P1)=1000$, $rC(P2)=1000$ bzw. $rC(P3)=0100$, $rC(P4)=0100$.

wenn $rC(P1) \& rC(P2) \neq 0$, dann liegt die Linie komplett außerhalb

Fall 2: Linie liegt komplett innerhalb, z. B. $rC(P1)=0000$, $rC(P2)=0000$.

wenn $rC(P1) | rC(P2) == 0$, dann liegt Linie komplett innerhalb

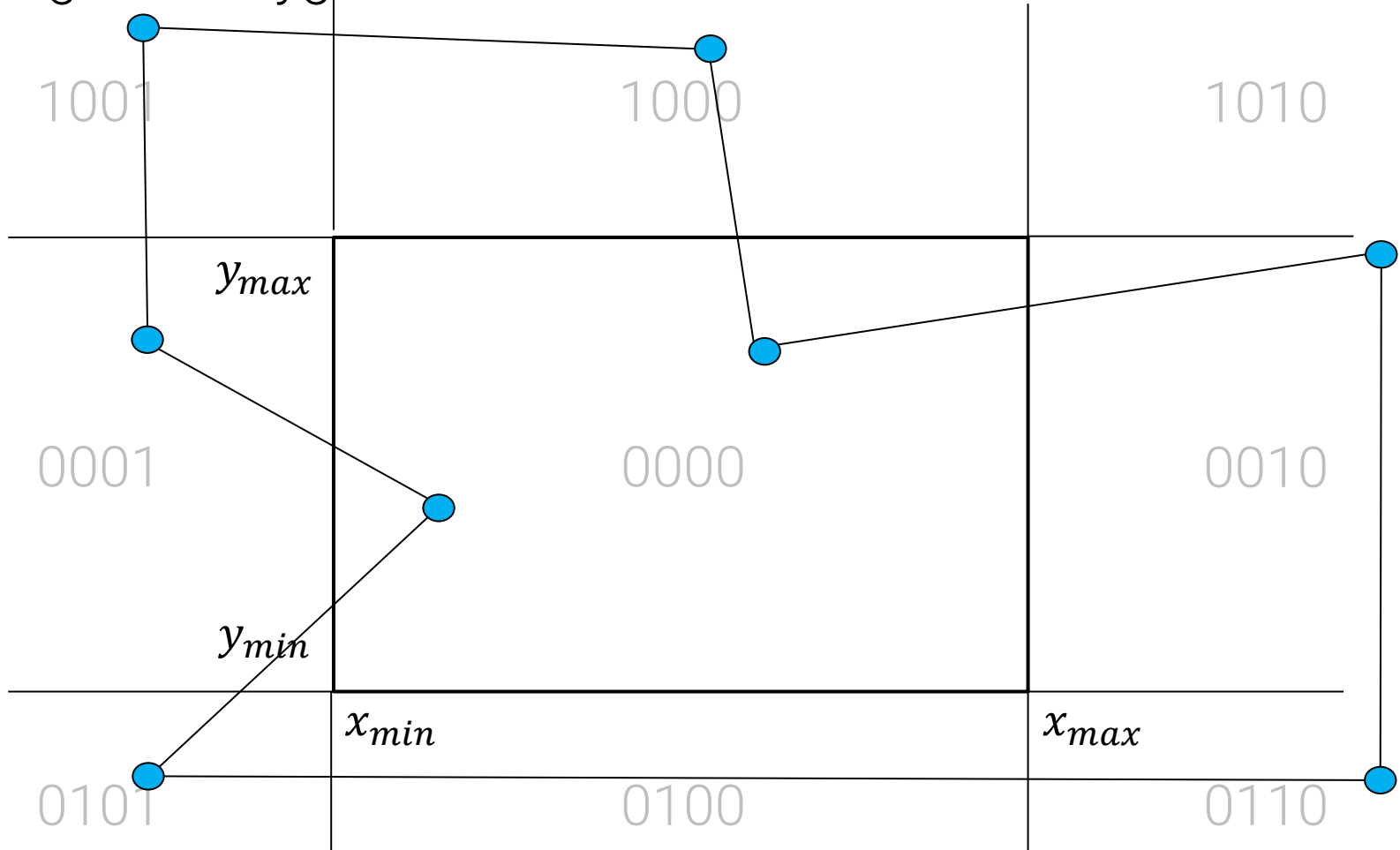
Fall 3: In allen anderen Fällen müssen wir Schnittpunkte mit $xMin$, $xMax$, $yMin$, $yMax$ finden und die neuen Abschnitte auf die ersten beiden Fälle zurückführen.



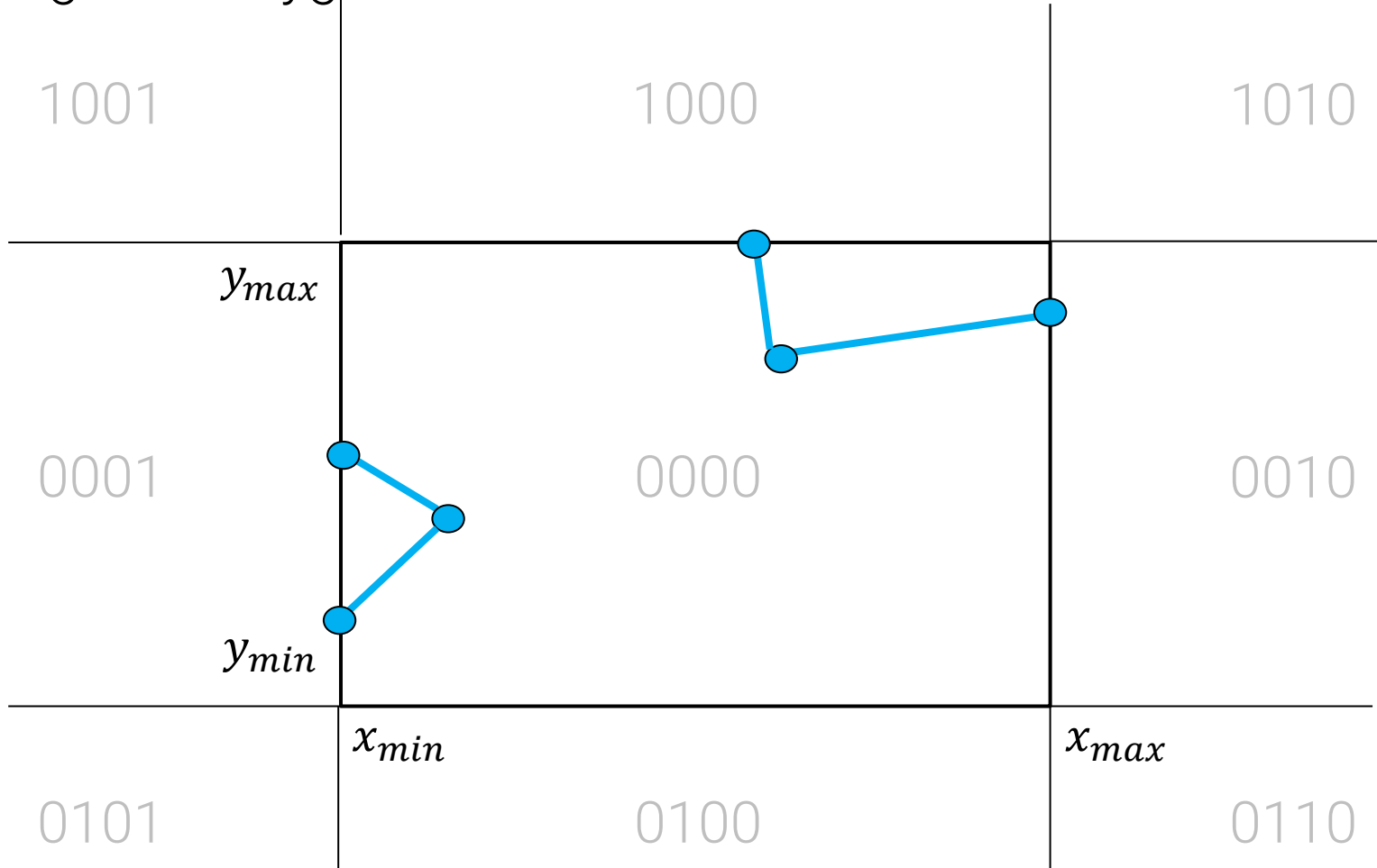


Jetzt wollen wir **Polygone** clippen

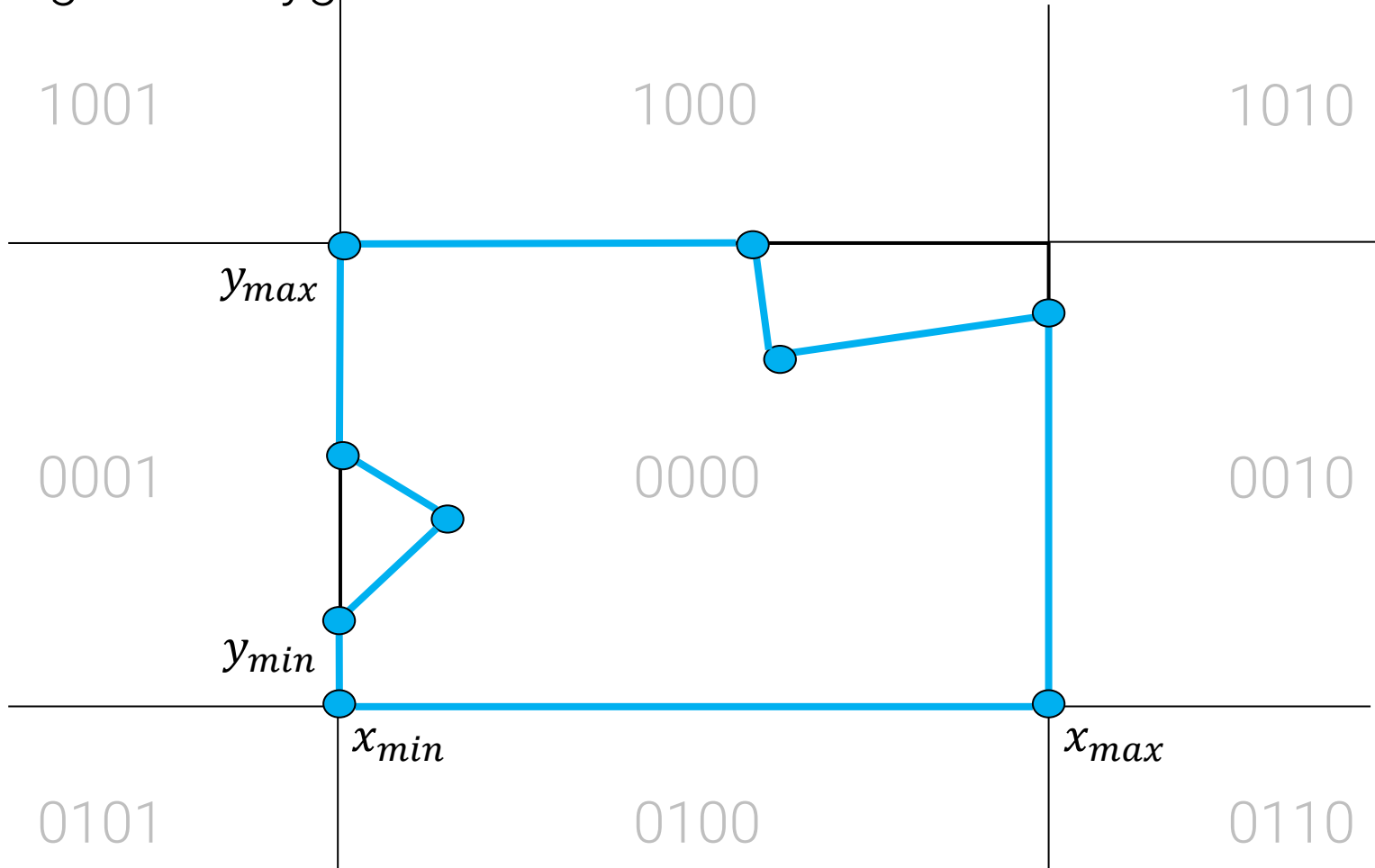
Clipping von Polygonen



Clipping von Polygonen

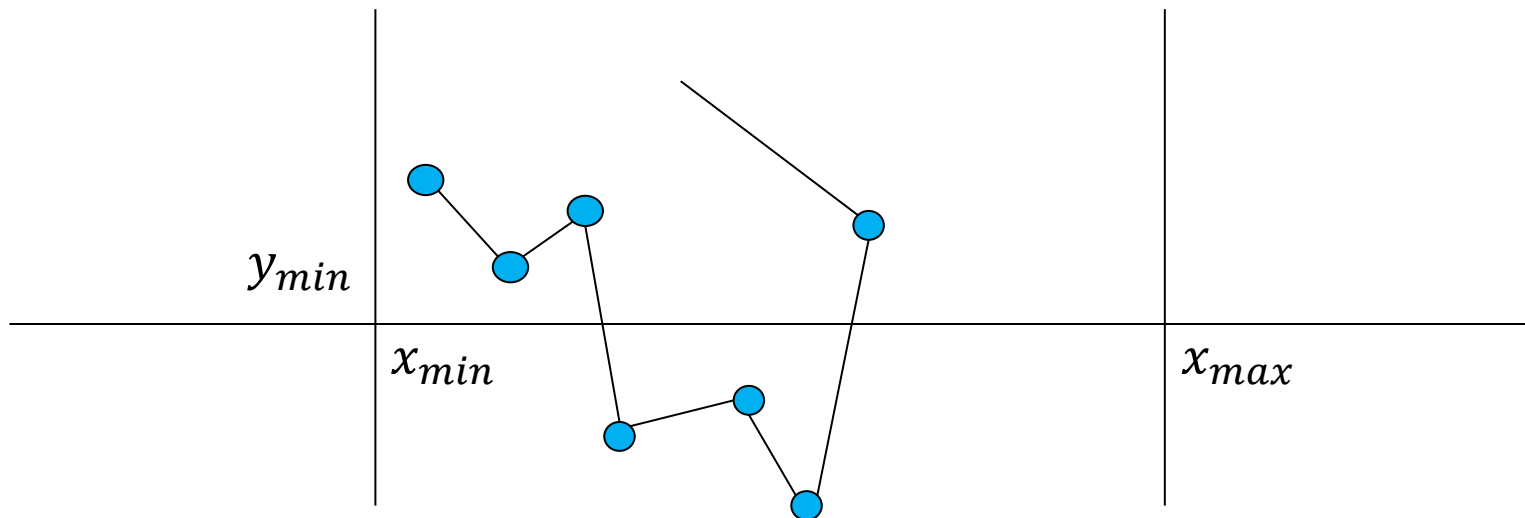


Clipping von Polygonen



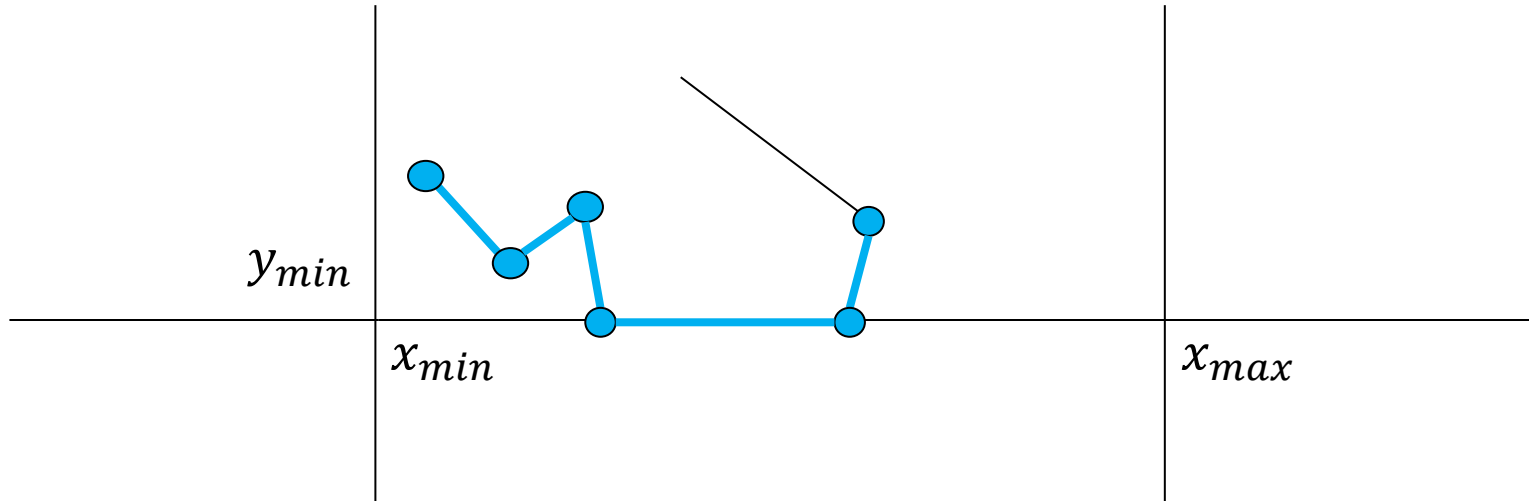
Lokale Betrachtung

Betrachten wir mal einen **einfachen Fall**, der sich nur an einer Clippingseite abspielt. Die Entscheidung für den Verbleib der Kanten, bzw. die Neuerstellung, lässt sich über eine Greedy-Strategie ableiten:



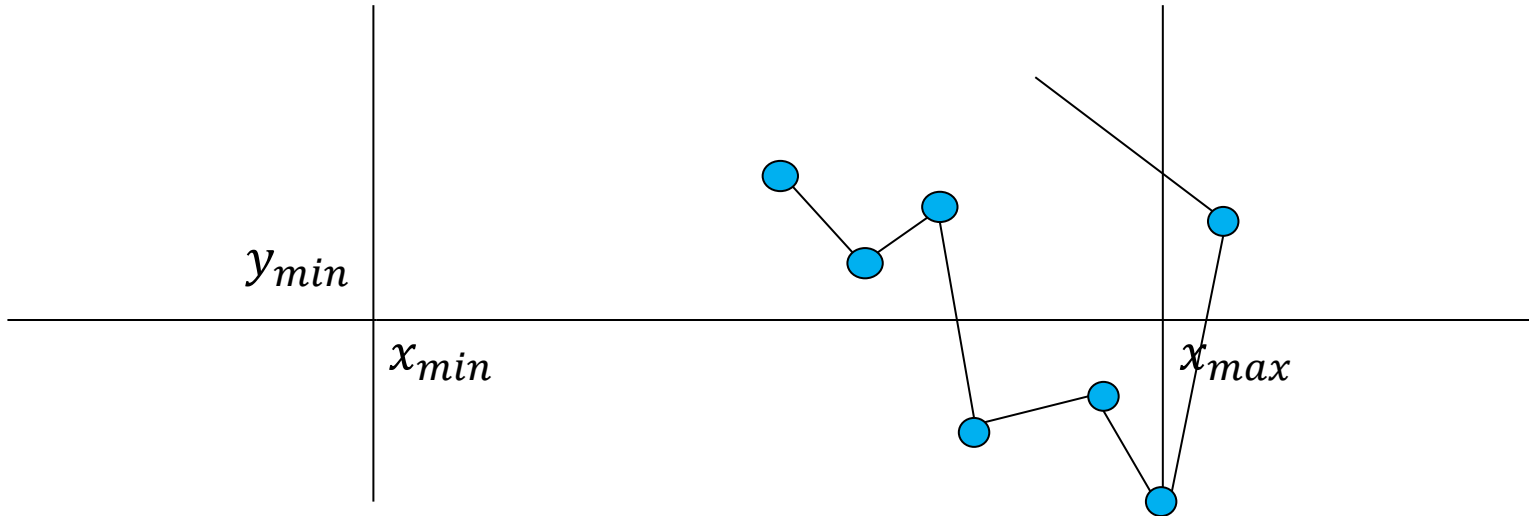
Lokale Betrachtung

Betrachten wir mal einen **einfachen Fall**, der sich nur an einer Clippingseite abspielt. Die Entscheidung für den Verbleib der Kanten, bzw. die Neuerstellung, lässt sich über eine Greedy-Strategie ableiten:



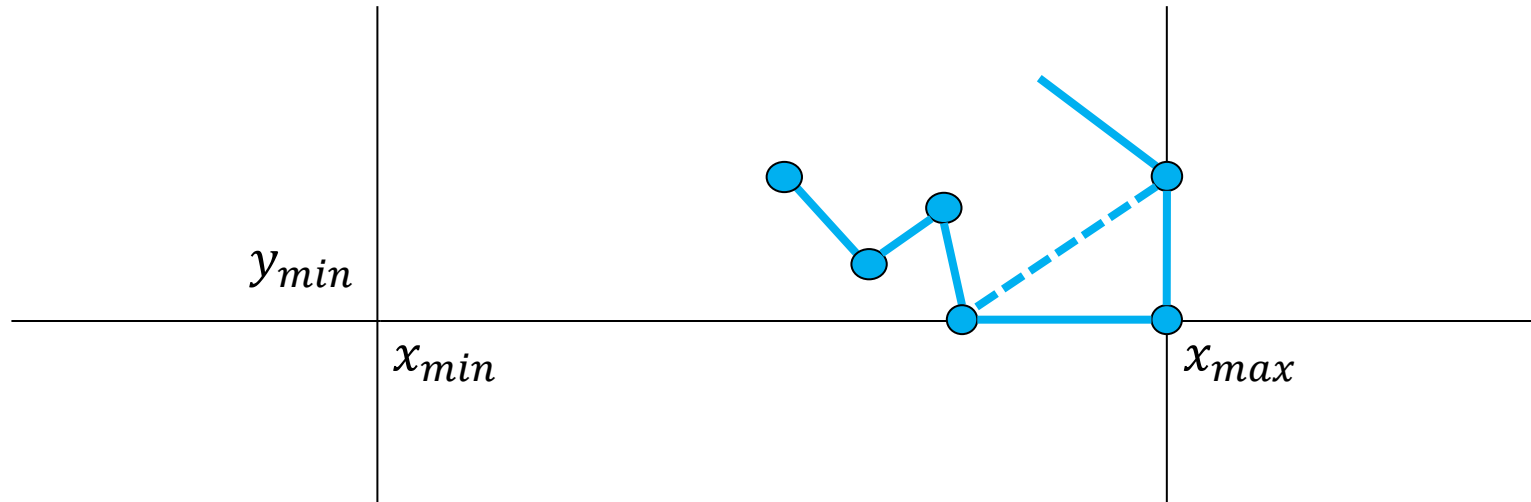
Lokale Betrachtung

Betrachten wir mal einen etwas **komplexeren Fall**, der sich an zwei Clippingseiten abspielt. Die Entscheidung für den Verbleib der Kanten, bzw. die Neuerstellung, lässt sich auch hier über eine Greedy-Strategie ableiten:



Lokale Betrachtung

Betrachten wir mal einen etwas **komplexeren Fall**, der sich an zwei Clippingseiten abspielt. Die Entscheidung für den Verbleib der Kanten, bzw. die Neuerstellung, lässt sich auch hier über eine Greedy-Strategie ableiten:

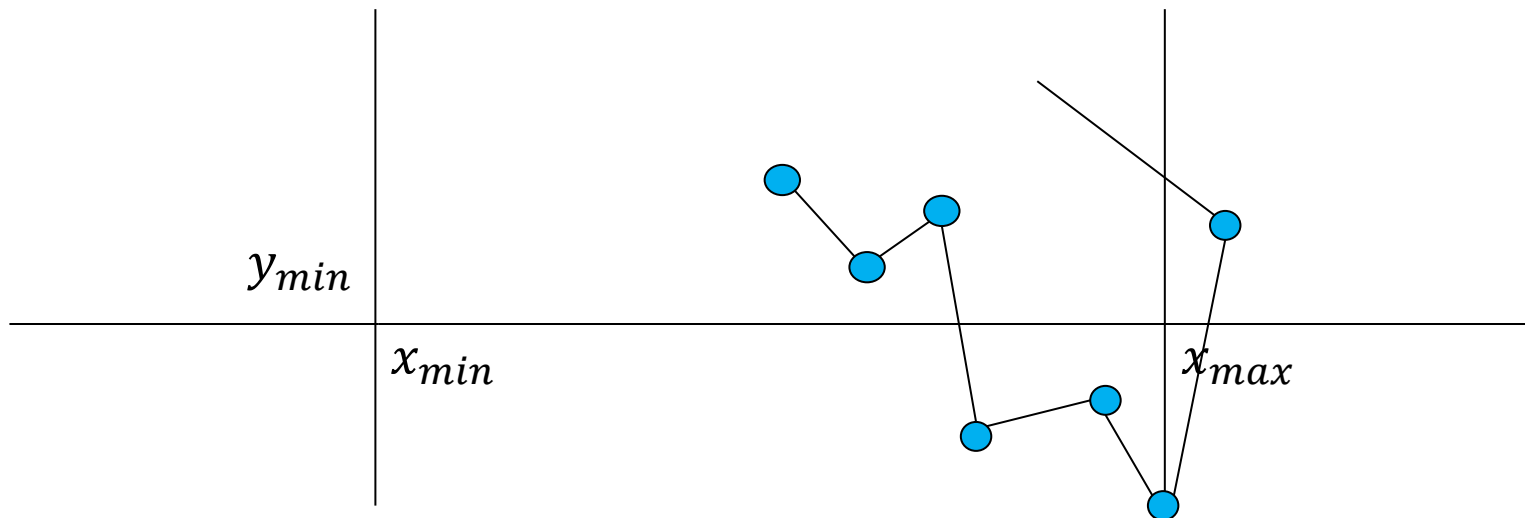


Idee von Sutherland&Hodgman

Jetzt müssen wir nur noch alle vier Seiten nacheinander clippen:

```
public void sutherlandHodgeman(Vector<Point> polygon) {  
    sutherlandHodgeman(polygon, xMin, 0);  
    sutherlandHodgeman(polygon, yMin, 1);  
    sutherlandHodgeman(polygon, xMax, 2);  
    sutherlandHodgeman(polygon, yMax, 3);  
}
```

Java

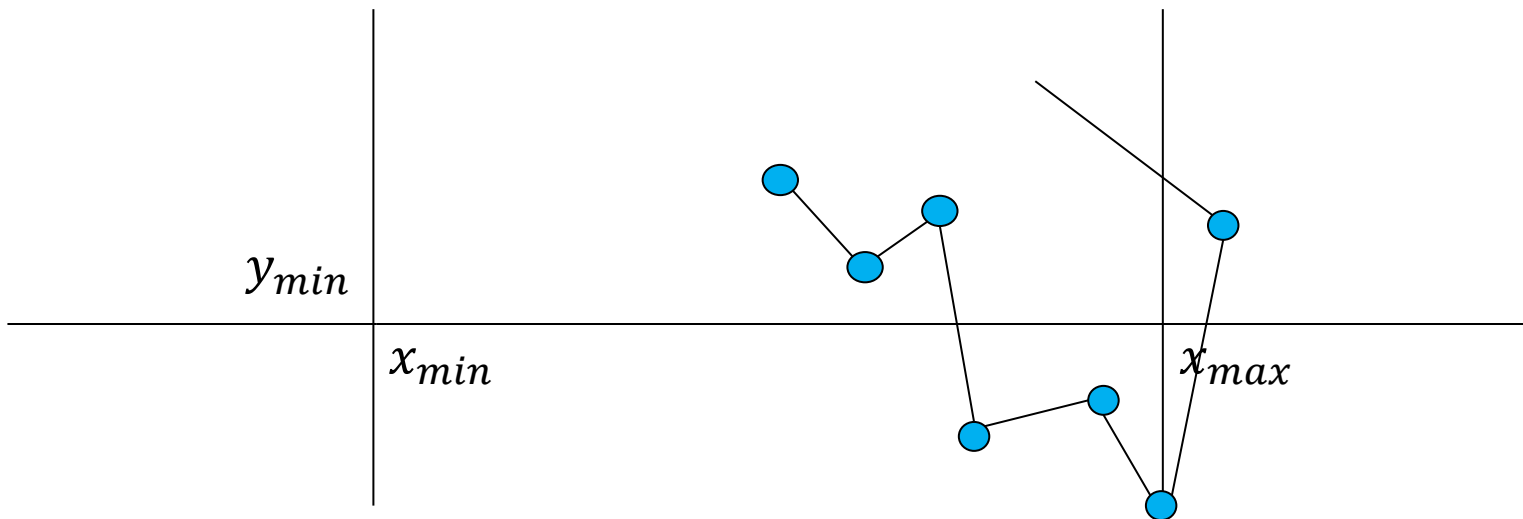


Idee von Sutherland&Hodgman

Jetzt müssen wir nur noch alle vier Seiten nacheinander clippen:

```
public void sutherlandHodgeman(Vector<Point> polygon) {  
    sutherlandHodgeman(polygon, xMin, 0);  
    sutherlandHodgeman(polygon, yMin, 1);  
    sutherlandHodgeman(polygon, xMax, 2);  
    sutherlandHodgeman(polygon, yMax, 3);  
}
```

Java

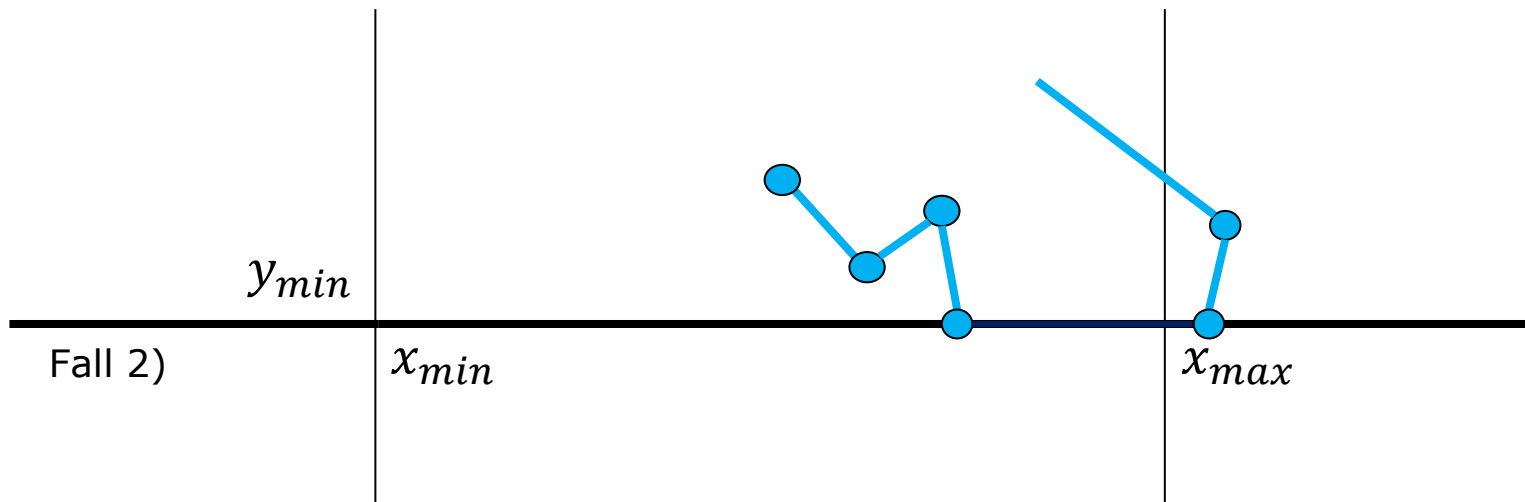


Idee von Sutherland&Hodgman

Jetzt müssen wir nur noch alle vier Seiten nacheinander clippen:

```
public void sutherlandHodgeman(Vector<Point> polygon) {  
    sutherlandHodgeman(polygon, xMin, 0);  
    sutherlandHodgeman(polygon, yMin, 1);  
    sutherlandHodgeman(polygon, xMax, 2);  
    sutherlandHodgeman(polygon, yMax, 3);  
}
```

Java

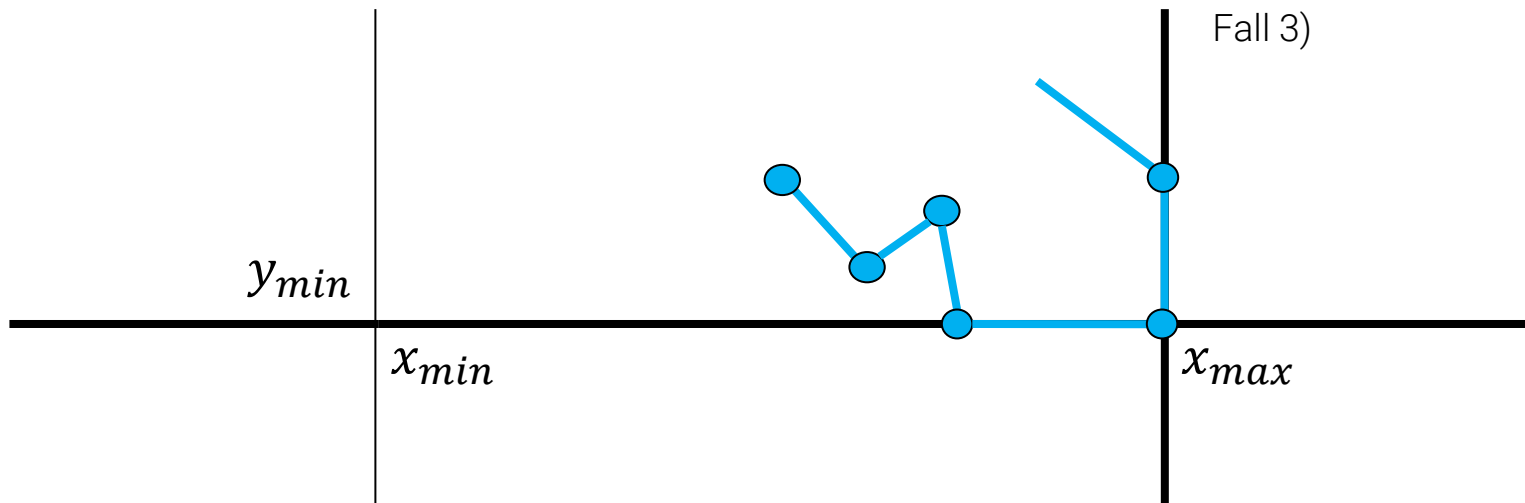


Idee von Sutherland&Hodgman

Jetzt müssen wir nur noch alle vier Seiten nacheinander clippen:

```
public void sutherlandHodgeman(Vector<Point> polygon) {  
    sutherlandHodgeman(polygon, xMin, 0);  
    sutherlandHodgeman(polygon, yMin, 1);  
    sutherlandHodgeman(polygon, xMax, 2);  
    sutherlandHodgeman(polygon, yMax, 3);  
}
```

Java



Implementierungsbeispiel – Sutherland-Hodgeman

Schauen wir uns hier das Beispiel in einer interaktiven Umgebung an. Es kann ein Polygon definiert werden.

Danach können wir das Sutherland-Hodgeman-Verfahren zum Clipping eines Polygons schrittweise durchführen.

Polygon: Klicke Gitterpunkte. Schließen durch Klick auf P0 (mind. 3 Punkte).
SPACE = nächster Sutherland-Hodgman-Schritt, B = zurück, A = Autoplay, R = Reset
Reihenfolge wie in der Vorlesung: LEFT -> BOTTOM -> RIGHT -> TOP
Nach dem Schließen: Punkte ziehen möglich; Clipping wird dann sauber zurückgesetzt.
Pro Schritt wird genau eine Kante S->P an genau einer Clippingseite verarbeitet.

Modus: Polygon zeichnen
aktuelle Seite: LEFT
Kantenindex: -
Schritte: 0
Fall: -

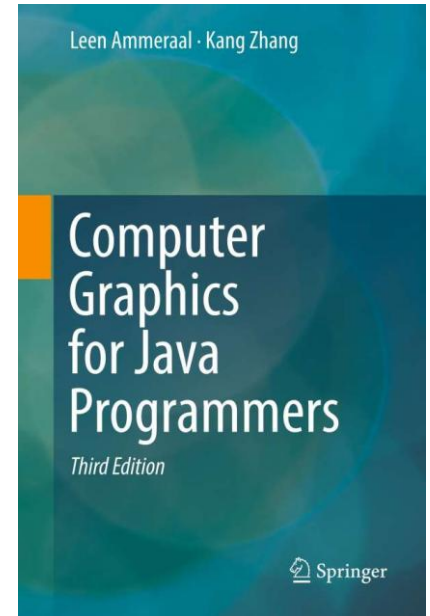
Clippingfenster

Legende:

- Originalpolygon
- aktuelles Eingabepolygon
- Ausgabepolygon im Aufbau
- aktive Clippingseite
- sichtbar / innen
- außen
- Schnittpunkt
- aktuelle Kante S->P

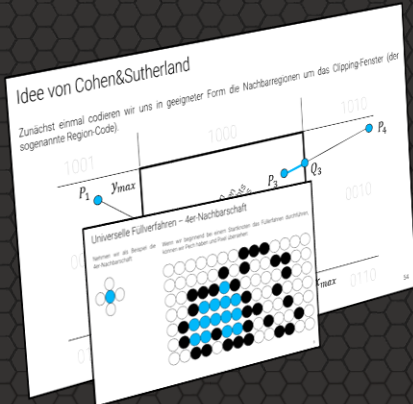
Literatur und weitere Code-Beispiele

Als weitere Literaturquelle für Computergrafik in Verbindung mit Java möchte ich das Buch von Ammeraal und Zhang empfehlen:



Rasterisierung von Flächen und Clipping von Linien und Polygonen

Key Takeaways



- Rasterbasierte Flächenfüllung
- Flood Fill basiert auf Pixelnachbarschaften
- Scan-Line Filling
- Clipping reduziert Geometrie vor Rasterisierung
- Cohen-Sutherland nutzt Region-Codes
- Sutherland-Hodgman clippt Polygone iterativ

Fragestellungen zum Vorlesungsteil

Im Anschluss an diese Veranstaltung sollten Sie die folgenden Fragen sicher beantworten können:

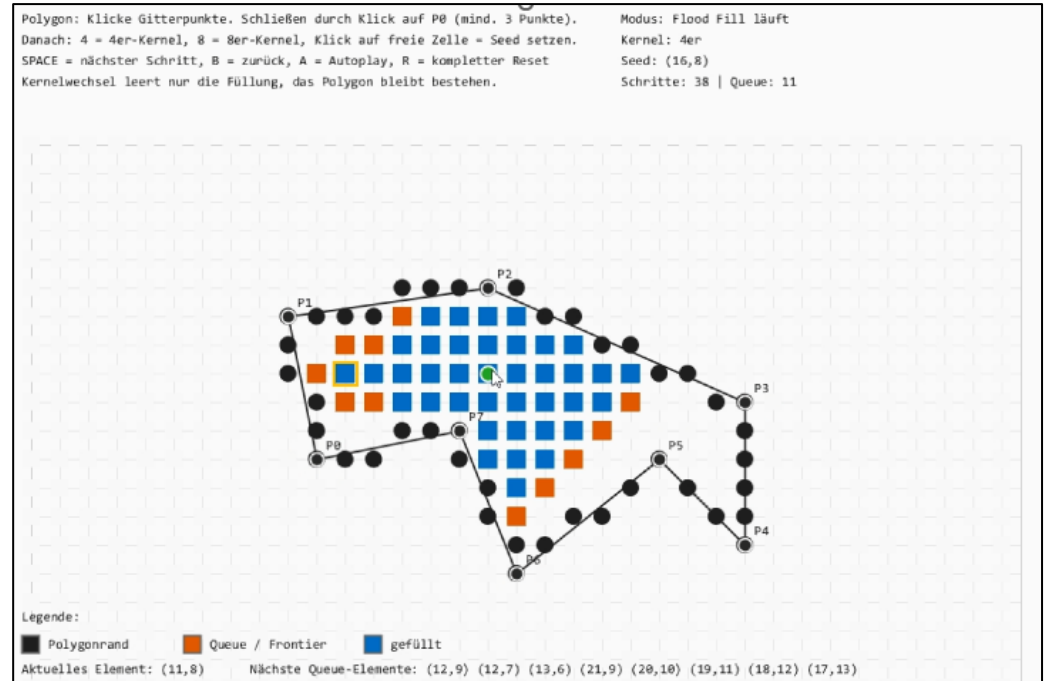
- (1) Beschreiben Sie die Abarbeitung universeller Füllverfahren und deren Schwächen.
- (2) Erläutern Sie das Scan-Line-Verfahren anhand eines Beispiels und die Besonderheit der Kantensortierung.
- (3) Wie lässt sich der Bresenham-Algorithmus zum Kreiszeichen für das Ausfüllen erweitern?
- (4) Erläutern Sie das Clipping von Linien und Polygonen. Welchen Einfluss hat hier der Region-Code?



Praktische Aufgaben

Folgende praktische Aufgaben sollten Sie jetzt selbstständig durchführen:

Aufgabe 1) [Processing] Implementieren Sie den Flood-Fill-Algorithmus mit einem variablen symmetrischen Kernel.





Vielen Dank für die Aufmerksamkeit